CLAUDE CODE FOR ACTUARIES

The Complete Guide

Life & Annuities • Property & Casualty • Reinsurance

Ethel Cédric Nzati, AQ
Actuary • Data Scientist

David Schraub, FSA, CERA, MAAA, AQ
Independent Actuary • AI & Model Governance

Table of Contents

Executive Summary

1. Introduction

- 1.1 Why This Guide Exists
- 1.2 What This Guide Is and What This Guide Is Not
- 1.3 How to Read This Guide
- 1.4 Companion Project Folders

2. Why Claude Code, and Not Just Claude

- 2.1 The Three Levels of AI Use

3. Installation and Setup

- 3.1 Prerequisites
- 3.2 Installation in Three Steps
- 3.3 Security Settings Before You Start
- 3.4 A Tour of the Interface
- 3.5 The CLAUDE.md File
- 3.6 Connecting to Your Data with MCP
- 3.7 Running a Companion Pack End to End

4. What Claude Code Can Do Beyond Code

5. Life & Annuities

- 5.1 Risk Division — ALM Team
- 5.2 Risk Division — Assets and Investment Strategy
- 5.3 Risk Division — Modeling Team
- 5.4 Risk Division — Risk Measurement and Capital
- 5.5 Model Validation and Actuarial Oversight
- 5.6 Valuation and Financial Reporting
- 5.7 Product Development
- 5.8 Corporate Finance and Investments
- 5.9 Health and Group Benefits

6. Property & Casualty

- 6.1 Pricing
- 6.2 Reserving
- 6.3 Performance, Reporting and Compliance

7. Reinsurance

8. Building an Internal Tool in a Day

9. Building AI Agents with Claude Code

© 2026 Necflow AI & David Schraub Actuarial Consultancy. All rights reserved.

9.1 What an Agent Is, Concretely

9.2 Two Families of Particularly Useful Agents

9.3 Other Agents Useful to an Actuary

10. Security, Confidentiality and AI Governance

11. Best Practices and the Correction Workflow

11.1 Prompting Claude Code Well

11.2 The Correction Loop

11.3 A Trick That Changes Everything: Synthetic Datasets

11.4 A Complete Example: the Quarterly Portfolio Review Pipeline

11.5 Going Further: Migrating the Whole Study to R

12. Appendix — A CLAUDE.md Template for Actuaries

About the Authors

© 2026 Necflow AI & David Schraub Actuarial Consultancy. All rights reserved.

Executive Summary

Actuaries spend a considerable share of their time on repetitive technical tasks or mundane work with little value added: writing Python, R or VBA code, cleaning portfolio data, producing regulatory reports, documenting models, testing developments. These are low-leverage tasks that consume scarce, expensive expertise.

Claude Code changes that equation. It is an AI agent that works directly in your environment: it writes code, runs it, reads its own errors and corrects itself until the result is right. A VBA valuation function that takes a day to develop and debug is generated in minutes. Year-end technical documentation that takes days is produced in one session. An ALM dashboard or an RBC calculator is built in a day, without waiting on IT.

This guide covers the full range of actuarial practice in North America: life and annuities (ALM, assets, modeling, risk measurement and capital, model validation, statutory valuation, product development, investments, health and group benefits), property & casualty (pricing, reserving, reporting) and reinsurance. Each use case is presented with its business context, a description of what Claude Code will design, an exhaustive ready-to-paste prompt, and the expected inputs and outputs.

Ten of these use cases ship with a downloadable companion project folder: the dataset, the CLAUDE.md, the ready-to-paste prompts, and reference results to check your run against. The full list, with download links, is in chapter 1.

Beyond business use cases, the guide shows how to build internal tools in a day, how to create AI agents able to answer your teams' questions about your models, and how to frame all of it with security, confidentiality and governance rules suited to insurance data and to the expectations of US and Canadian regulators.

> **Note:** The goal of this guide is not to do AI for AI's sake. It is to make you faster and more rigorous on the tasks that matter, while keeping the actuary at the center of every decision.

© 2026 Necflow AI & David Schraub Actuarial Consultancy. All rights reserved.

CHAPTER 1

Introduction

1.1 Why This Guide Exists

Day to day, a large part of actuarial work is technical execution: building models in Python or R, reconciling portfolio extracts, maintaining complex Excel workbooks, debugging legacy VBA, and testing developments against functional specifications.

Claude Code turns these tasks into simple instructions. You describe what you want in plain English, and Claude Code builds it, runs it, reads its own errors and corrects itself until the result is right.

This guide is organized by practice area and by team. Whether you are an ALM actuary, an auto pricing actuary, a modeler, a reserving actuary or a reinsurance specialist, you will find use cases directly applicable to your day-to-day work, with prompts ready to copy and paste.

1.2 What This Guide Is and What This Guide Is Not

This guide focuses on Claude Code as a practical tool to support actuarial model development. It demonstrates how actuaries can use Claude Code to translate business and modeling requirements into working code and accelerate portions of the development process.

This guide is not a guide to model risk management, actuarial governance, or regulatory compliance. Actuaries using Claude Code should continue to operate within established professional, regulatory, and organizational frameworks. Relevant guidance includes the International Actuarial Association (IAA) standards, particularly ISAP 1A – Governance of Models and ISAP 1 – General Actuarial Practice. In the United States, ASOP 56 – Modeling provides important guidance regarding the development, use, validation, and governance of models. Regulatory requirements may also apply, including VM-30, VM-G, SR 26-2, OSFI E-23, the NAIC AI Model Bulletin, Colorado SB21-169, the AMF Model Risk Management Guideline, DFS Circular Letter 2025-7, and any internal governance standards established by an organization. Additional guidance may be available from actuarial organizations such as the SOA, CAS, AAA, CIA, and other professional bodies.

This guide is also not a guide to software engineering best practices. Topics such as coding standards, code optimization, software architecture, testing strategies, cybersecurity, version control, and long-term maintainability are largely outside its scope. While these considerations remain important, they have been extensively documented elsewhere.

Similarly, this guide does not attempt to address many of the core elements of actuarial modeling, including business problem formulation, model architecture, assumption development, validation frameworks, controls, documentation requirements, change management, or the exercise of professional judgment. These remain essential components of the model lifecycle and should be addressed through appropriate governance processes.

Although this document is titled the Complete Guide, the objective is practicality and initial start rather than comprehensiveness or implementation of a company/country specific interpretation. The guide intentionally focuses on the effective use of Claude Code and does not provide exhaustive treatment of

© 2026 Necflow AI & David Schraub Actuarial Consultancy. All rights reserved.

prompt engineering, token optimization, model configuration, or other highly technical aspects of large language models. Nor does it seek to comprehensively address broader principles such as transparency, explainability, auditability, traceability, and reproducibility.

Most importantly, this guide should never be interpreted as a substitute for actuarial expertise or professional judgment.

A common misconception is that a single, perfectly crafted prompt can fully capture both stated and unstated requirements and automatically produce a flawless solution. In practice, model development is an iterative process requiring expertise, critical review, testing, and refinement. Claude Code is a powerful assistant, but it is not a replacement for the knowledge, judgment, and accountability of the actuary.

To keep this document accessible and practical, simplifications and shortcuts are used throughout. Readers should recognize these simplifications and apply appropriate professional judgment when adapting the concepts to real-world situations. Ultimately, responsibility for the design, implementation, validation, governance, and appropriateness of any actuarial model remains with the actuary, not the tool.

1.3 How to Read This Guide

Every use case follows the same structure: the business context, a description of what Claude Code will design, a detailed ready-to-use prompt, and the expected inputs and outputs. The prompts are deliberately exhaustive: copied as-is, they should produce a usable result.

> **Tip:** The prompts in this guide are starting points. The more context you add about your own environment (file names, conventions, data structures), the more precise the result will be on the first iteration.

A word on data. The prompts are written against data your company already holds, so they name inputs (a portfolio extract, a zero-coupon curve, a loss triangle) without printing their layout. Each use case's "Expected inputs" list tells you what the prompt presumes; when the business context is not enough to see what a file or a parameter represents, the companion packs close the gap: each pack's CLAUDE.md carries the full data dictionary — every file, every column, every parameter, and how they map to the prompt's functions. For your own data, do the same: describe the layout in your CLAUDE.md (the chapter 12 template has a section for it), and the prompt will land on the first try.

1.4 Companion Project Folders

Several use cases in this guide come with a downloadable companion project folder: the CLAUDE.md, the ready-to-paste prompts and the dataset. Look for the green "Companion resources" boxes. Download the folder, open it in Claude Code, paste the prompt, and replicate the use case end to end on your own machine. Some datasets keep their original file and column names (occasionally in French, as noted in each pack): leave them unchanged so the prompts run as-is.

Pack	Where in the guide	What you get	Link
Bond valuation library	Ch. 5.1 — Use case 1	15-bond synthetic portfolio, zero-coupon and default curves, reference values	Download
Bond projection — Excel/VBA & Streamlit	Ch. 5.3 — Use case 8	Bond table, projected risk-free curves, functional specification, two exercises	Download

© 2026 Necflow AI & David Schraub Actuarial Consultancy. All rights reserved.

Pack	Where in the guide	What you get	Link
NAIC RBC calculator	Ch. 5.4 — Use case 13	Exposures and factors workbook, hand-checked reference table	Download
Pandemic data visualization	Ch. 5.9 — Use case 23	Three public datasets (hospital, vaccination, economic) — the recommended first project	Download
Auto pricing — GLM vs machine learning	Ch. 6.1 — Use cases 24-25	freMTPL2freq, 678,000 public motor policies (CASdatasets)	Download
Reserving — Chain Ladder	Ch. 6.2 — Use case 26	Ten-year cumulative auto claims triangle, plain Chain Ladder	Download
Reserving — CL, BF and Mack	Ch. 6.2 — Use case 26	Two triangles with premiums and a priori loss ratios, reference results for the three methods	Download
AI agent — documentation & code	Ch. 9	Complete Streamlit + ChromaDB chat application, sample document-and-code corpus	Download
Quarterly portfolio review pipeline	Ch. 11	500,000-policy synthetic portfolio with planted data issues, four chained prompts	Download
Python-to-R migration	Ch. 11	Worked migration of the quarterly-review scripts, reconciliation outputs	Download

Chapter 3 ends with a step-by-step walkthrough, with annotated screenshots, of running a pack in Claude Code from scratch (section 3.7).

> **Companion resources:** all ten packs are also gathered in a single folder: Google Drive folder.

© 2026 Necflow AI & David Schraub Actuarial Consultancy. All rights reserved.

CHAPTER 2

Why Claude Code, and Not Just Claude

Claude is an artificial intelligence model developed by Anthropic, able to understand natural language, reason, and generate text as well as code. Most actuaries know it through the chat interface: you ask a question, paste some code, get an answer. Claude Code is different. It is an agent, built on the same model, that works directly inside your working environment.

Three features make the difference:

Autonomy

Claude Code does not stop at writing code. It executes it, reads the error messages in the console, understands what went wrong, and fixes itself until the script works. This feedback loop (write, test, correct) happens without you copy-pasting errors manually.

Local access to your files

Claude Code can read an entire model folder: multiple Python, R or VBA files, and understand the dependencies between them before proposing a change. The web chat only sees what you paste into the conversation. For an actuarial model spread across dozens of files, that difference is decisive.

End-to-end execution

Claude Code can create files, run calculations, install libraries, generate charts and produce reports, all in a single session. You go from a request in plain English to a concrete deliverable without leaving your workspace.

> **Note:** In short: the web chat answers questions. Claude Code completes tasks. For an actuary who works with models spread across many files and needs executable results, that changes everything.

2.1 The Three Levels of AI Use

In practice, actuaries adopt AI in three stages. Knowing where you stand helps you see where you are going.

- **Level 1 — Code generation.** “Write a Python function to calculate expected deaths.” You get code, you paste it, you test it. Useful but limited: the AI knows nothing about your project.
- **Level 2 — Project understanding.** Claude Code reads your files, understands your existing scripts, modifies and runs them. It works with context. This is where most of this guide operates.
- **Level 3 — Workflow automation.** Read the data, validate it, run the analysis, generate the charts, produce the report: one instruction executes a whole reproducible pipeline. Chapter 11 shows a complete example.

© 2026 Necflow AI & David Schraub Actuarial Consultancy. All rights reserved.

CHAPTER 3

Installation and Setup

3.1 Prerequisites

- **Subscription:** an active Claude subscription (Claude Pro, about $20/month, is enough to start; no API key required).
- **A computer:** Mac, Windows or Linux, able to run the Claude Desktop app.
- **Internet:** a stable connection.
- **Skills:** none beyond your actuarial work. No terminal, no package manager, no system administration.

> **Note:** There are numerous ways to access Claude Code, with pricing linked to fixed-cost plans with usage limits or variable-cost plans (API access) without limits. The fixed-cost subscription is the simplest way to start; API access suits intensive or automated workloads.

3.2 Installation in Three Steps

Step 1 — Download the Claude Desktop app. Go to claude.ai/download and install the application for your operating system (Mac, Windows or Linux).

Step 2 — Create an account and subscribe. Sign up with your email address and subscribe to Claude Pro ($20/month).

Step 3 — Start working. Open Claude Code from the app, point it to your project folder (the working directory), and type your first instruction. That's it.

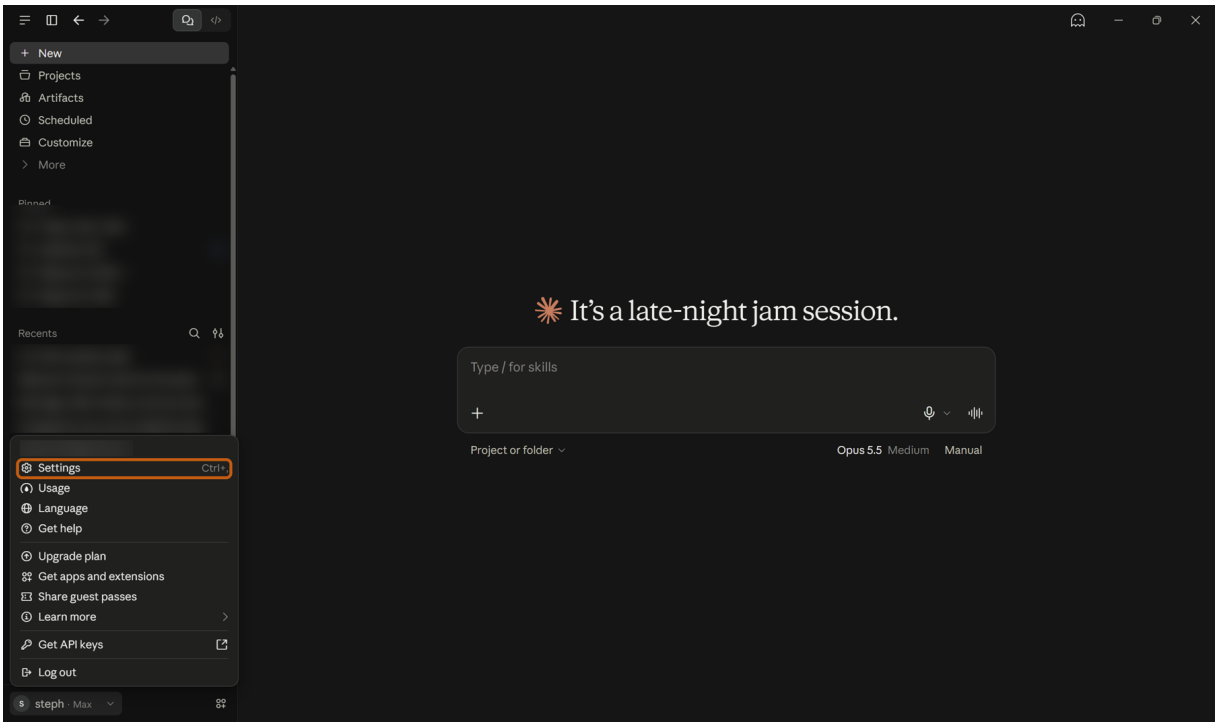
> **Note:** A second installation path exists: a command-line install through npm and Node.js, aimed at developers. It is not needed here: the desktop app ships with everything included, which matters in corporate environments where installing developer tooling is restricted.

3.3 Security Settings Before You Start

Before pointing Claude Code at anything, two reflexes. The first is scoping: give it access to the project folder only, never to a drive or folder tree holding documents you do not intend to share with the tool — the trust dialog shown in section 3.7 enforces exactly this. The second is deciding what may be done with your conversations. On individual plans, one setting controls whether your chats and coding sessions can be used to train Anthropic's models. Here is how to review it and turn it off:

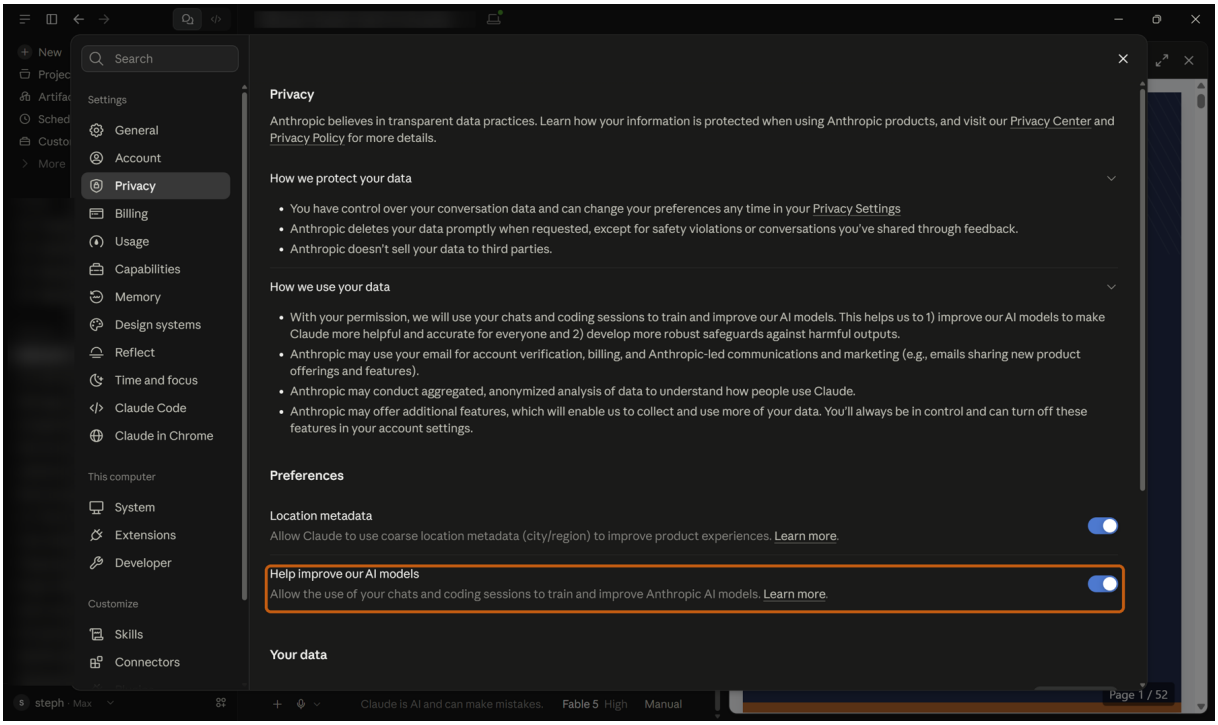
1. Open the account menu at the bottom left of the app and click Settings.

© 2026 Necflow AI & David Schraub Actuarial Consultancy. All rights reserved.



Step 1 — the account menu, bottom left, opens the Settings.

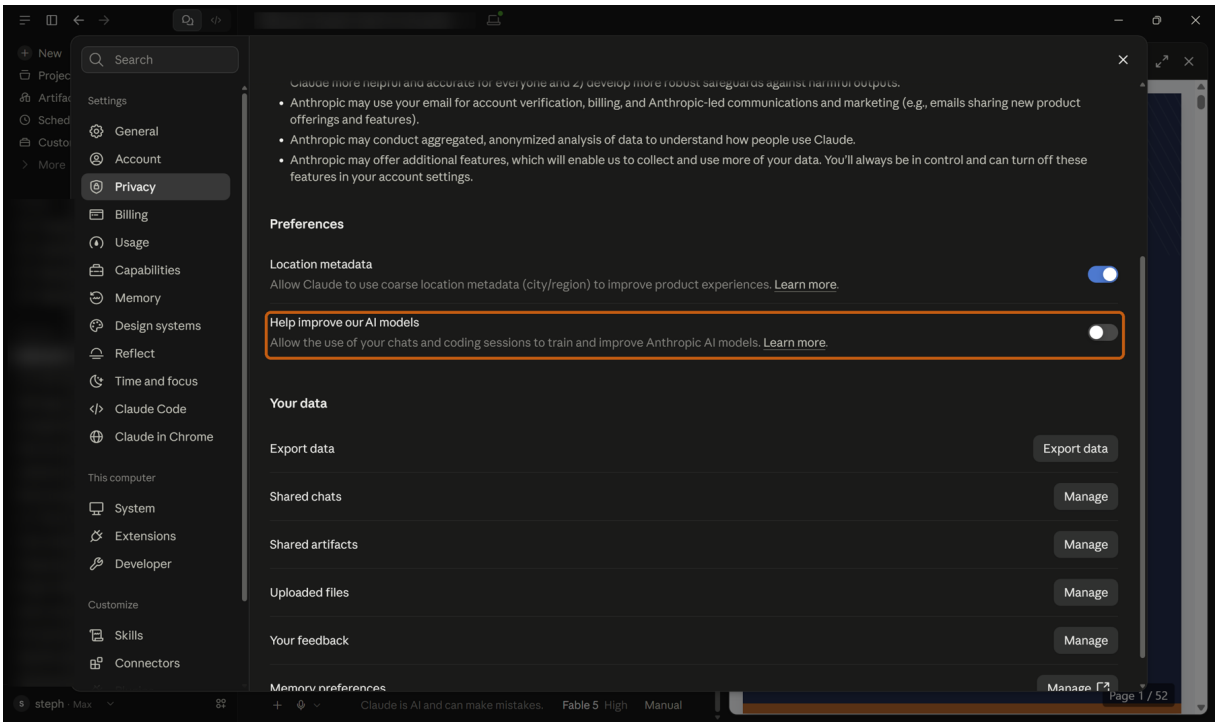
2. Go to the Privacy section. Under “How we use your data”, the policy states what the setting covers.



Step 2 — the Privacy section spells out how your data is used; the training setting is under Preferences.

3. Turn off “Help improve our AI models”. From that point on, your conversations and coding sessions are not used for model training.

© 2026 Necflow AI & David Schraub Actuarial Consultancy. All rights reserved.



Step 3 — "Help improve our AI models" turned off: conversations and coding sessions are no longer used for training.

In a corporate setting, these account-level settings complement — and never replace — the review by your IT department. Chapter 10 comes back to this.

A first test to check that everything works:

```
claude

> Create a file hello_actuary.py that builds a mortality table
> from a Gompertz law (B=0.00003, c=1.1), computes the associated
> life expectancy at age 40, and prints the result. Run it.
```

When you are ready for a first real project, start with the pandemic data-visualization use case in chapter 5.9: it ships with a complete downloadable project folder (data, CLAUDE.md, prompt) and makes an ideal first end-to-end run.

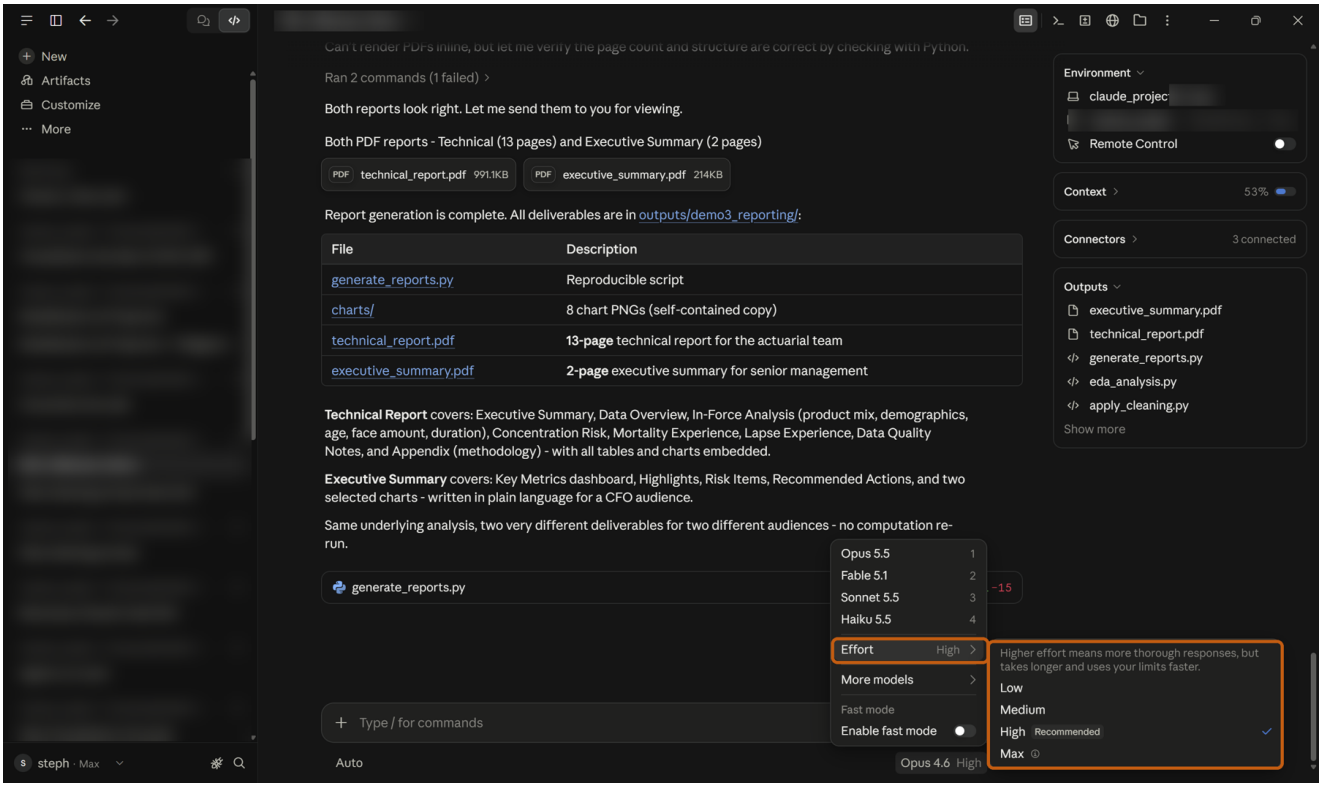
3.4 A Tour of the Interface

Five elements are worth knowing before your first real project:

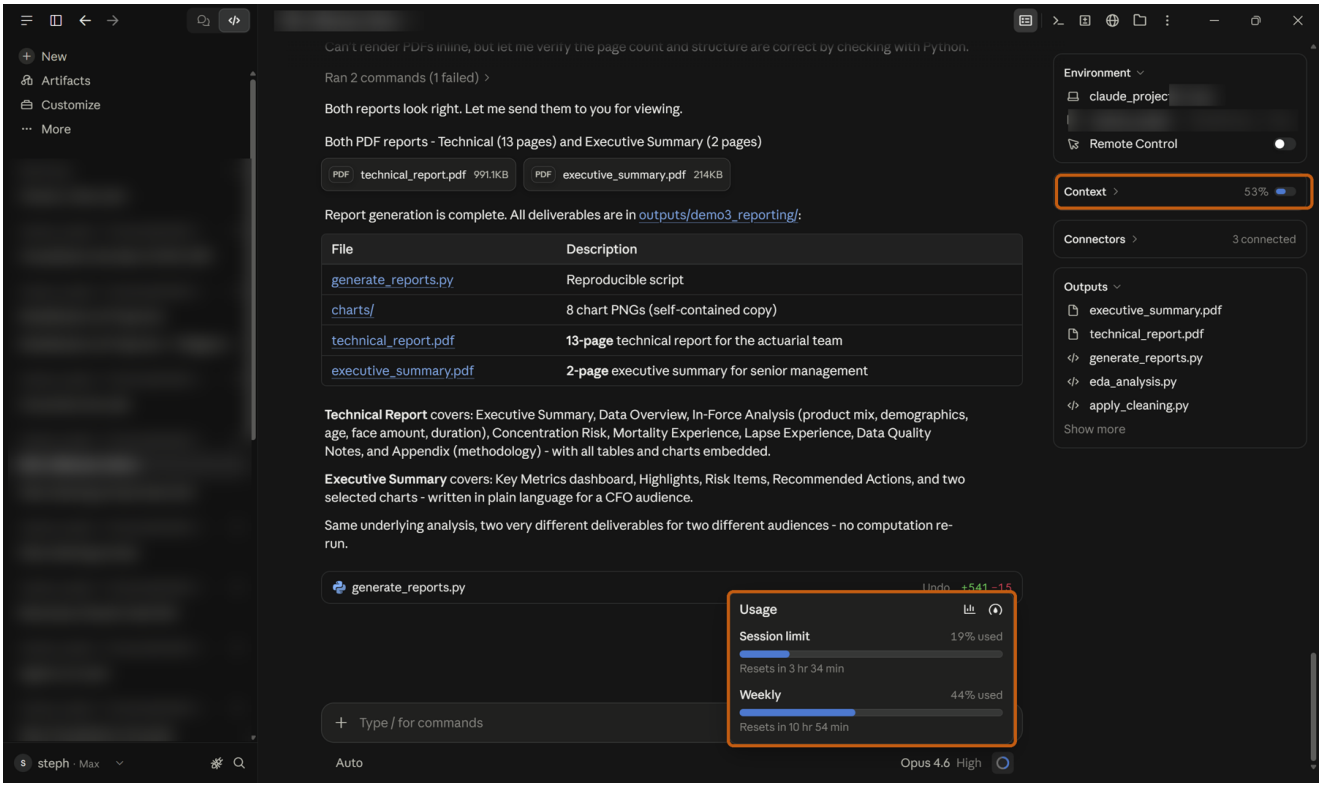
- **The chat zone:** where you converse with the agent in natural language. Instructions, follow-ups, corrections: everything goes through here.
- **Models:** several models are available, from the fastest and cheapest to the most capable. As a rule of thumb: use a mid-tier model for most actuarial work, and reserve the top tier for complex architecture or subtle reasoning. Model names evolve; the fast/balanced/powerful tiering remains.
- **Effort level:** controls how deeply the model thinks before answering. Raise it for complex modeling questions, keep it standard for routine tasks.
- **Context window:** how much of your project Claude can hold in mind at once. Large models read hundreds of pages; structure your project so the relevant files are easy to find.

© 2026 Necflow AI & David Schraub Actuarial Consultancy. All rights reserved.

- **Working directory:** the project folder you give Claude Code access to. It can read every file in it — which is exactly what makes it useful.



The effort level, in the model menu: higher effort means more thorough responses, at the price of time and usage.



The context panel (right) shows how much of the window the session has used; the Usage panel tracks your session and weekly limits.

© 2026 Necflow AI & David Schraub Actuarial Consultancy. All rights reserved.

> **Tip:** Recommendation: start every project by creating a dedicated folder (data, specs, outputs) and give Claude Code that folder as its working directory. Clean structure in, clean results out.

3.5 The CLAUDE.md File

This is the most important file of your setup. Claude Code reads it automatically at the start of every session. It contains your project context, your technical stack, your conventions and your business rules. Well written, it saves you from repeating the same instructions and dramatically improves the quality of the results.

Think of it as the briefing document you would hand a new analyst joining your team: where the data lives, which tools to use, which rules to follow, what never to do. A complete CLAUDE.md template for actuaries is provided in the appendix (chapter 12).

> **Tip:** Keep your CLAUDE.md under 200 lines. Claude Code re-reads it constantly; a bloated file dilutes the signal. Treat it as an index that points to detailed files, not as a manual.

3.6 Connecting to Your Data with MCP

MCP (Model Context Protocol) is a standard that lets Claude Code plug directly into your data sources instead of going through manual exports. Concretely, it is a connector: once configured, Claude Code can query a local SQL database, an Excel workbook, a document repository or an internal API, at the source.

Instead of hand-exporting a CSV of your portfolio to give to Claude Code, you configure the connector once, and Claude Code queries the data directly. For an actuary, this means a dashboard or a monitoring tool that reads live data, with no intermediate manipulation.

> **Tip:** Start simple: a well-written CLAUDE.md already delivers 80% of the value. MCP configuration comes later, when you want to industrialize data access.

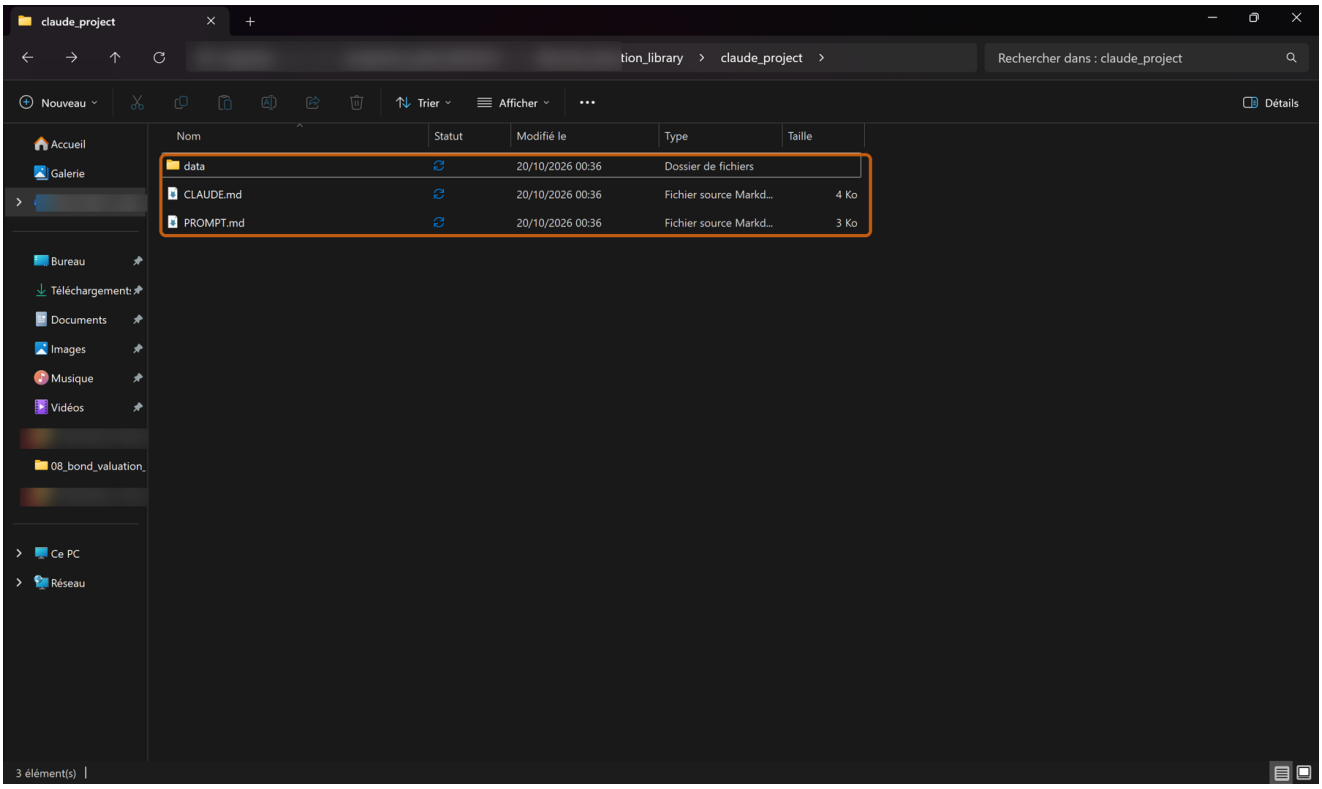
3.7 Running a Companion Pack End to End

Every companion pack of this guide (the table in chapter 1) runs with the same routine. This walkthrough uses the bond valuation library pack as the example; the steps are identical for the other nine — and for your own projects, since a pack is nothing more than a ready-made working folder.

Step 1 — Create the working folder on your computer

Download the pack from its Drive link into a dedicated folder on your machine. It already holds everything the session needs: the data in data/, the CLAUDE.md, and the PROMPT.md. For a project of your own, this is the step where you gather every resource that is useful and relevant: documentation, data extracts, existing code.

© 2026 Necflow AI & David Schraub Actuarial Consultancy. All rights reserved.



The working folder: data/, CLAUDE.md and PROMPT.md — everything Claude Code will need.

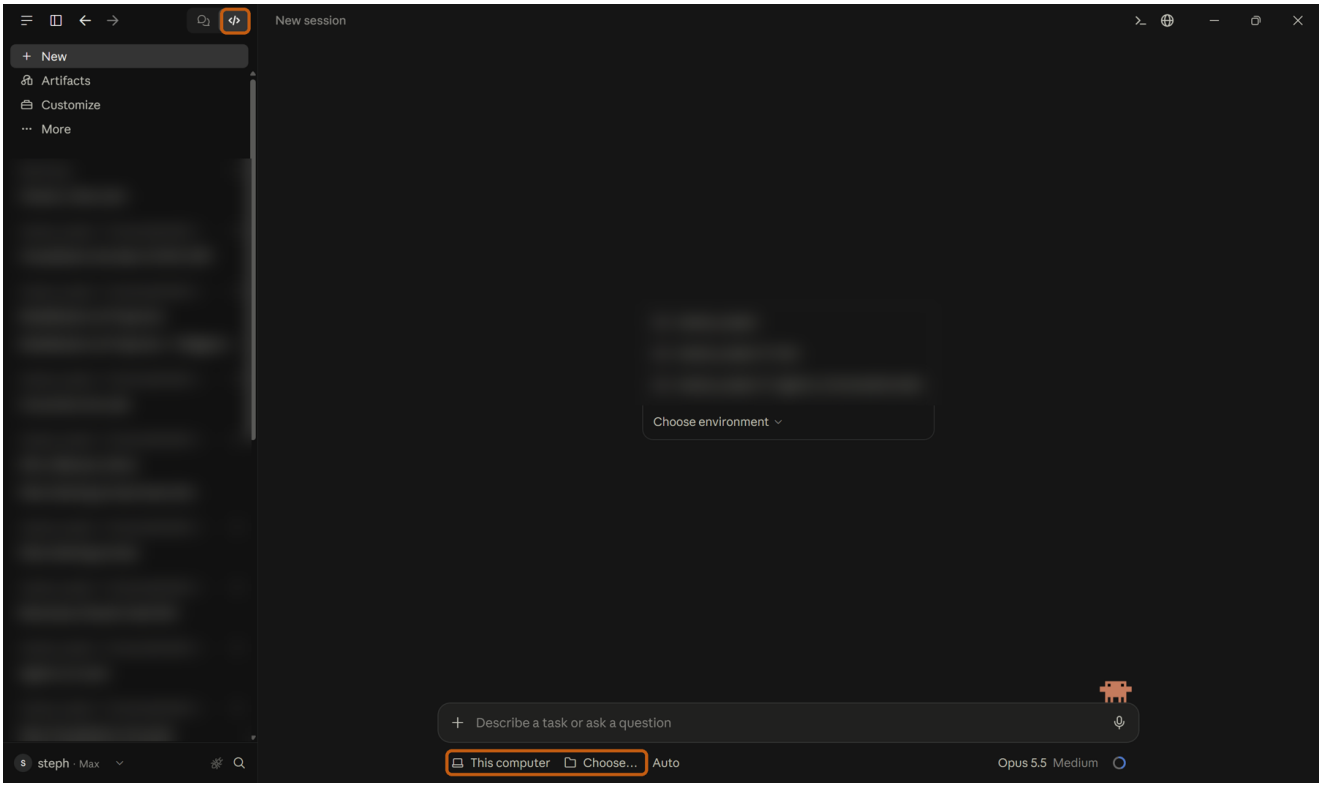
Step 2 — Create or adapt the CLAUDE.md and PROMPT.md

The packs ship with both files ready to use. For your own projects there are two ways to produce them. If you are starting out, describe your project to an AI assistant — Claude, ChatGPT or any other; in our experience Claude gives the best results for this exercise — and ask it to generate the two files, then read them critically and correct what needs correcting. Once you are more experienced, write them yourself, starting from the CLAUDE.md template in chapter 12. Note that this is the only step of the routine that happens in a chat interface rather than in Claude Code.

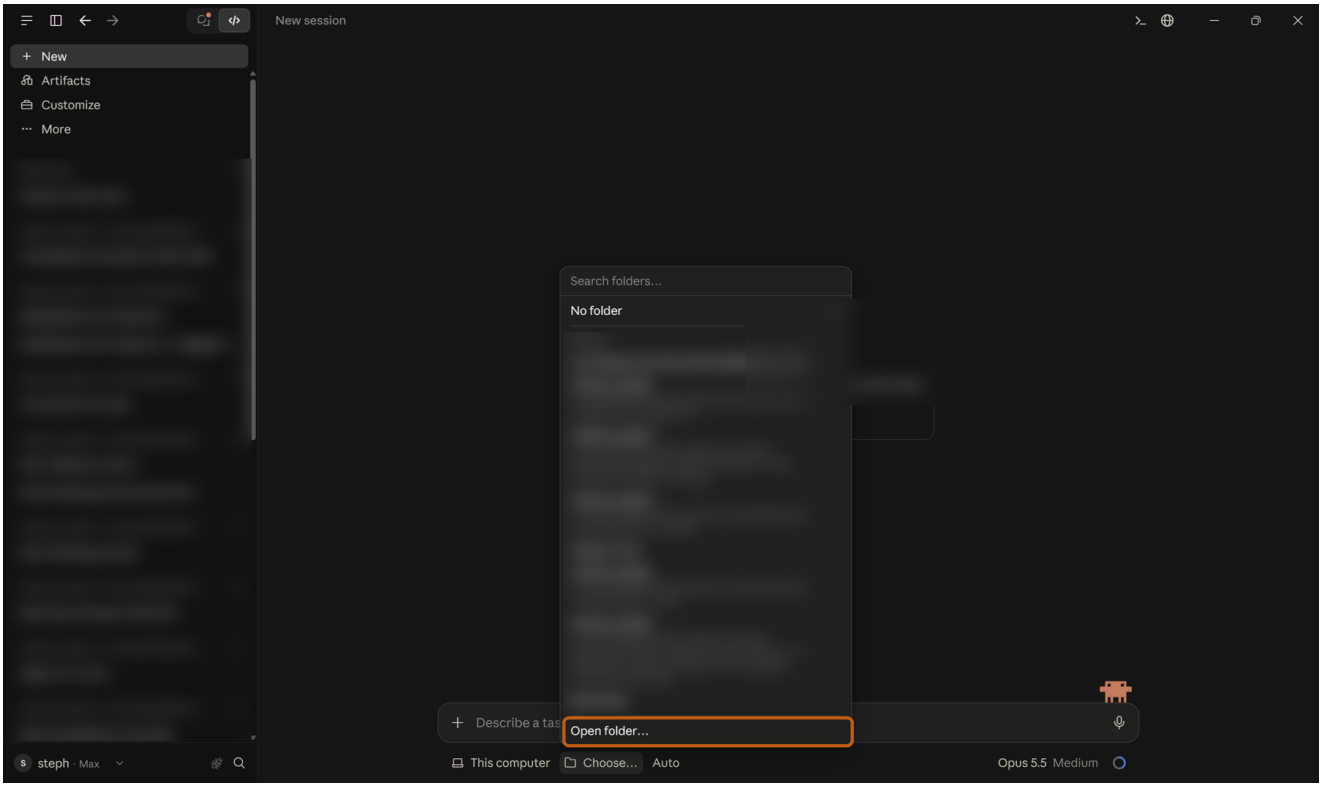
Step 3 — Open Claude Code and point it at the folder

In the Claude desktop app, switch to the Code side with the </> toggle at the top of the sidebar and click New. At the bottom of the input bar, select This computer, then Choose…, then Open folder…, and pick the pack's claude_project folder.

© 2026 Necflow AI & David Schraub Actuarial Consultancy. All rights reserved.



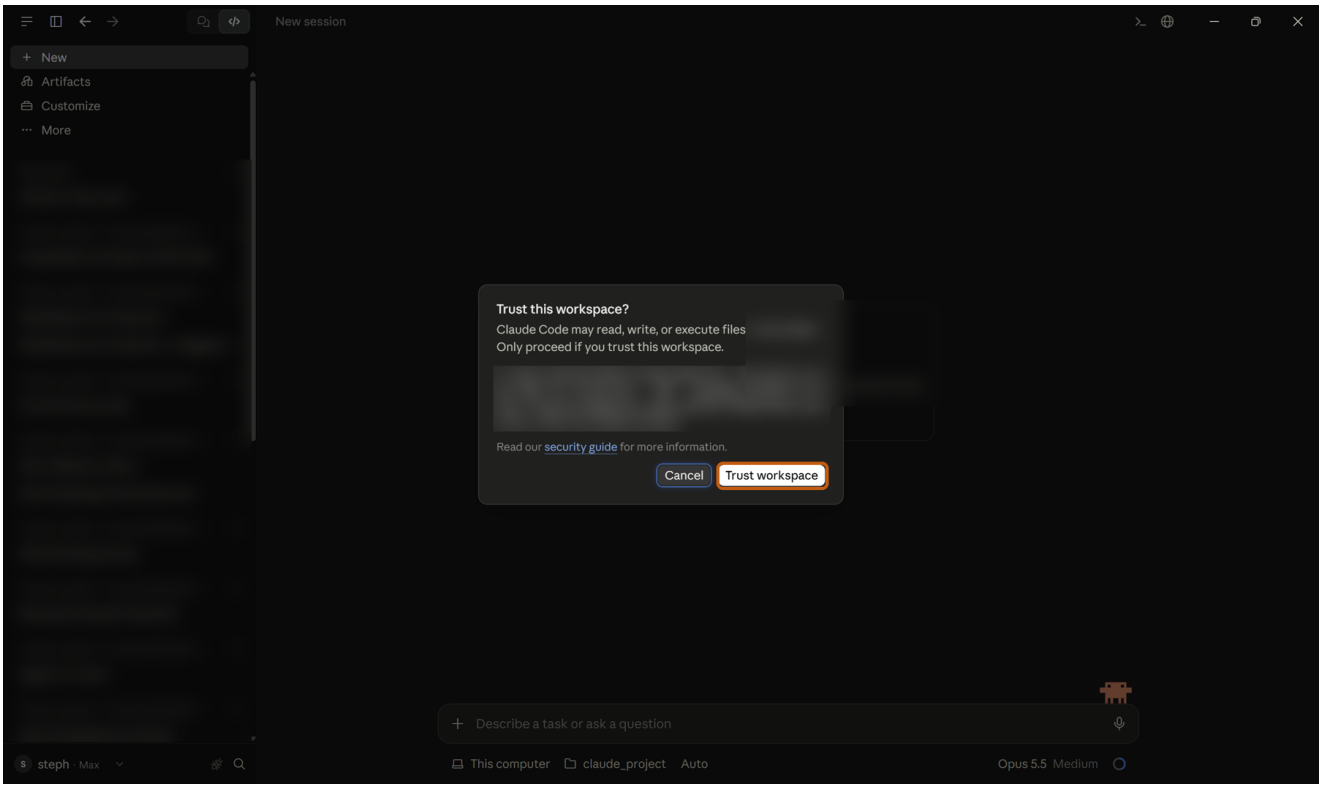
The Code side of the Claude desktop app: the </> toggle (top left) and the folder selector (bottom).



Open folder… lets you browse to the pack folder you just downloaded.

Claude Code then asks you to trust the workspace: it may read, write and execute files in that folder. Check that the path is the folder you intend — and only that folder — and that you trust its content, then click Trust workspace. This is the security boundary discussed at the start of this chapter: scope it deliberately.

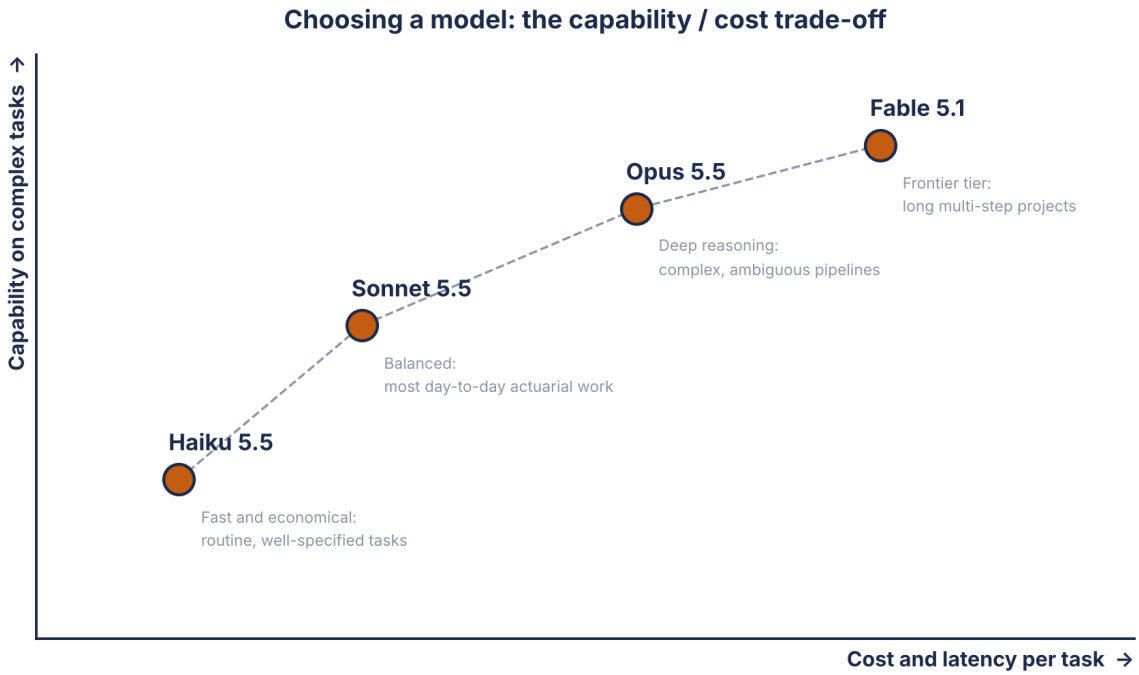
© 2026 Necflow AI & David Schraub Actuarial Consultancy. All rights reserved.



The trust prompt: Claude Code works only inside the folder you approve.

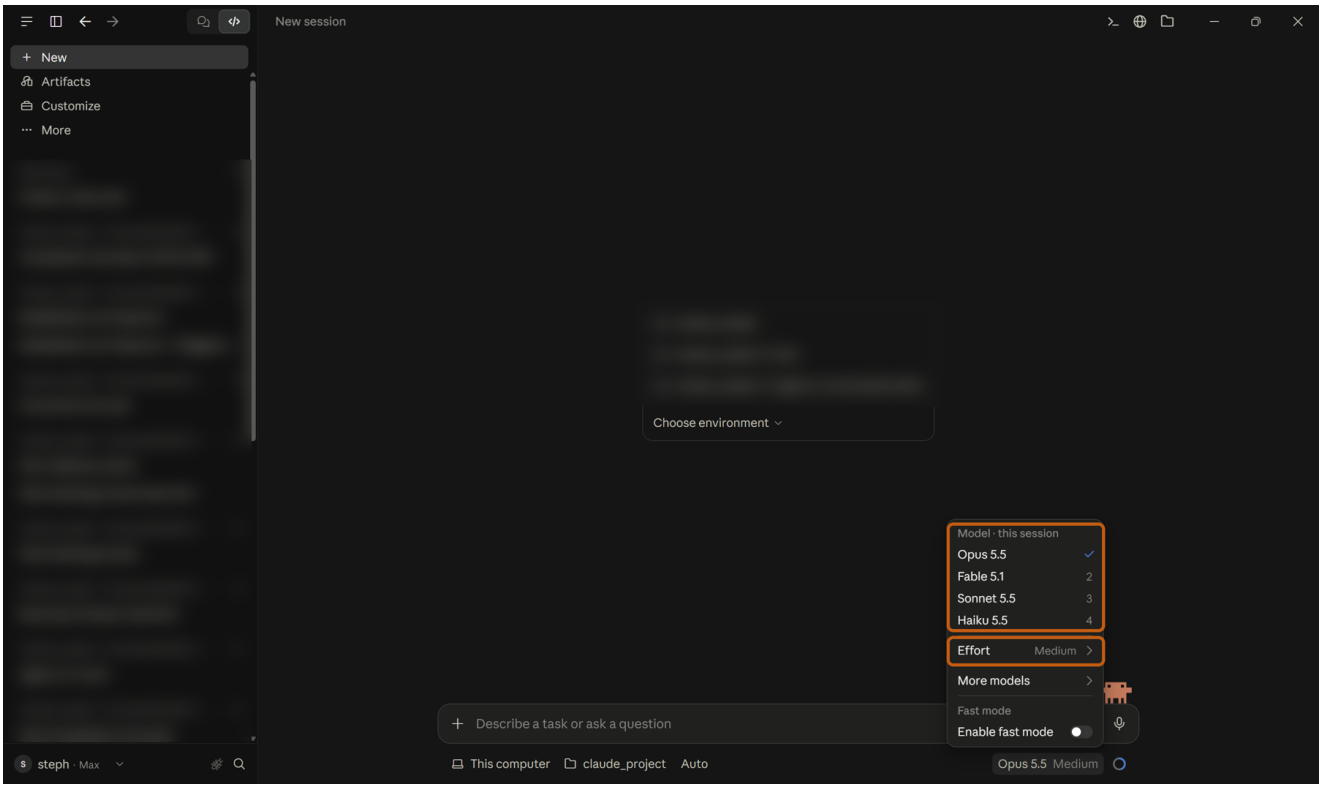
Step 4 — Pick the model and the effort level

The selector at the bottom right of the input bar sets the model and the effort level for the session. The trade-off is capability against speed and cost: the largest models reason deepest on long, ambiguous pipelines; lighter models are faster and cheaper and handle well-specified tasks very well; the effort setting tunes how long the model thinks before it acts. There is no single right choice — it depends on the task, and the right reflex is to experiment. For a first run of a pack, the defaults are fine.



Indicative positioning of the current Claude models — capability against cost and latency. Line-ups evolve; the trade-off and the tiering remain.

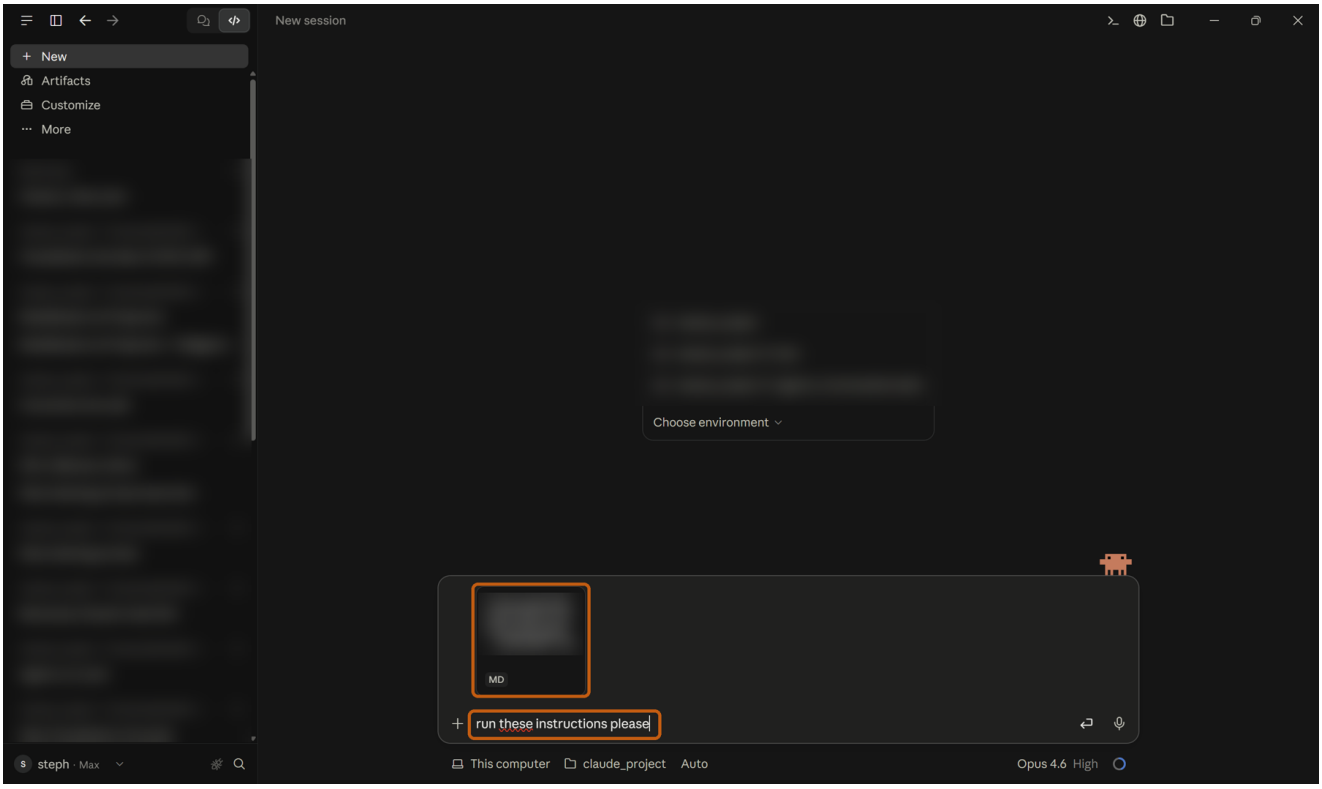
© 2026 Necflow AI & David Schraub Actuarial Consultancy. All rights reserved.



Model and effort are set per session, from the selector at the bottom right.

Step 5 — Run the prompt

Attach the PROMPT.md file to the input (or paste its content), type *run these instructions please*, and send.

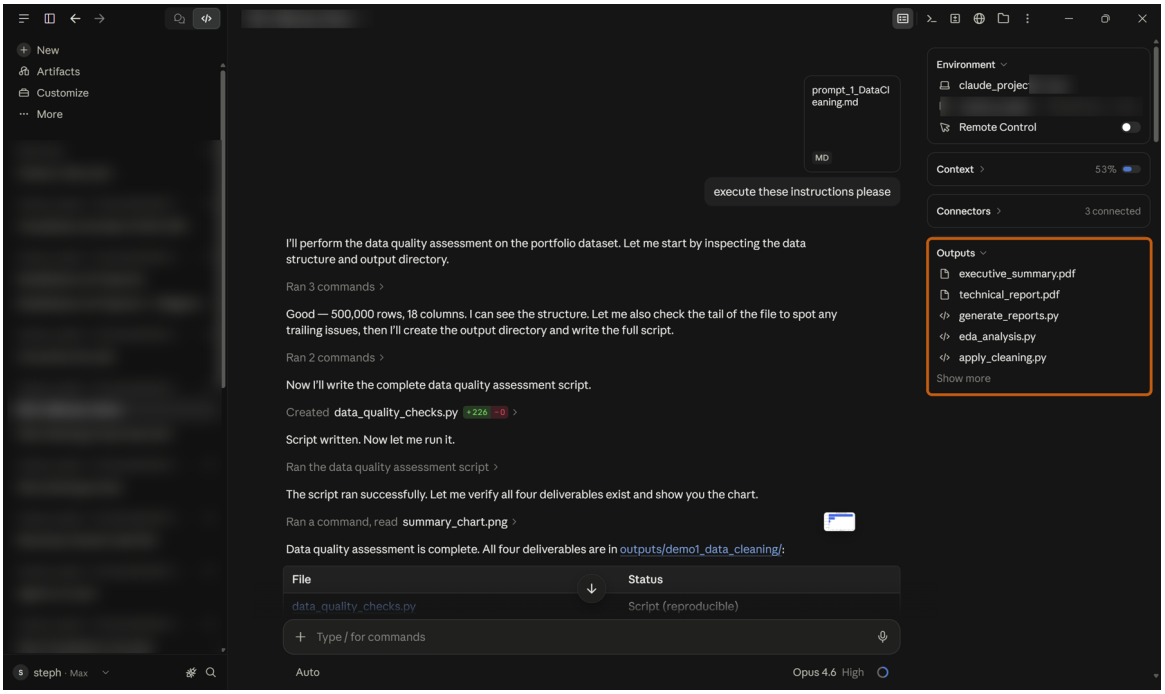


The PROMPT.md attached, the instruction typed: the session is ready to launch.

© 2026 Necflow AI & David Schraub Actuarial Consultancy. All rights reserved.

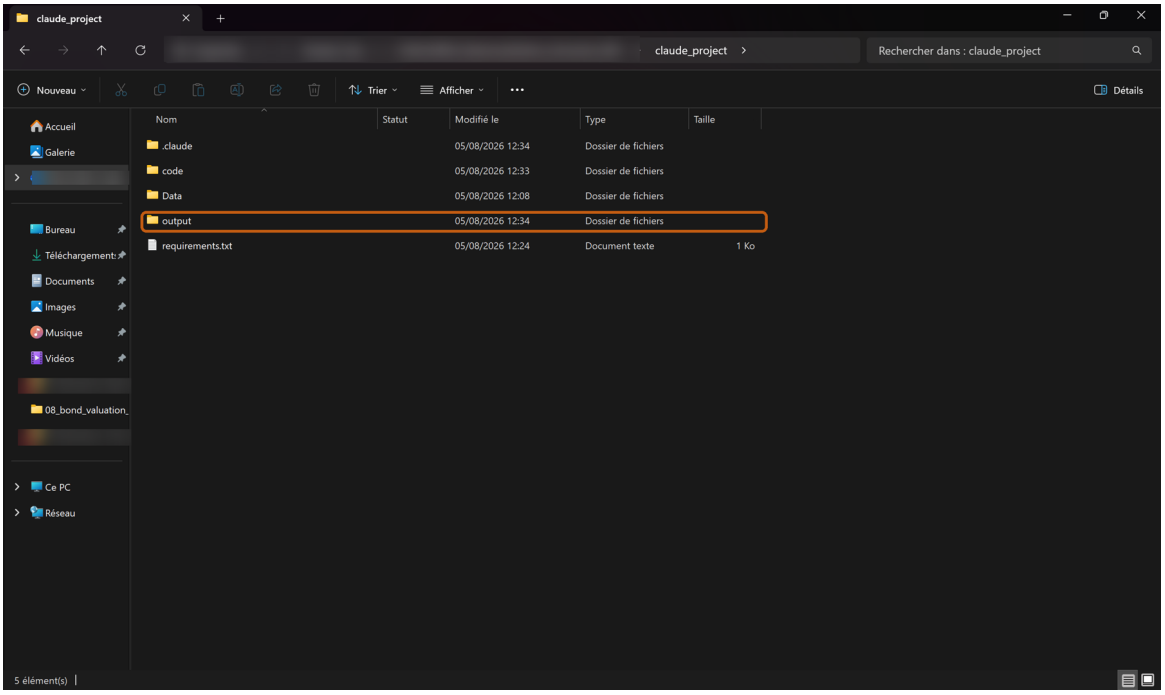
Step 6 — Let Claude Code work

Claude Code reads the CLAUDE.md, works through the prompt's steps, and may ask you to authorize a few actions along the way (running a command, installing a package). Review and approve them. When the work is done it tells you, and every deliverable is in your working folder — also listed in the Outputs panel on the right.



The session at work: progress in the center, deliverables collected in the Outputs panel.

And back in the working folder on your computer, the deliverables are simply there: the generated code, the output subfolder with the charts and reports, and the requirements.txt to reproduce the environment.



The working folder after the run: code, output and requirements.txt have been generated next to the data.

© 2026 Necflow AI & David Schraub Actuarial Consultancy. All rights reserved.

CHAPTER 4

What Claude Code Can Do Beyond Code

Reducing Claude Code to a code generator would be a mistake. For an actuary, it is also a versatile technical assistant. Here is what it does day to day, outside of pure code writing:

- **Spec review:** re-read a functional specification and check its business consistency
- **Gap detection:** detect inconsistencies or unhandled cases in a methodology
- **Test plans:** generate a structured test plan from a specification
- **Legacy code explanation:** explain, line by line, an old VBA model nobody understands anymore
- **Language conversion:** convert VBA to Python or R while preserving the business logic
- **Documentation:** write a model's technical documentation, LaTeX formulas included
- **Presentations:** prepare a PowerPoint deck from a model's results
- **README:** generate a clear README for an actuarial project
- **Architecture:** propose a software architecture for a new tool
- **Optimization:** speed up a projection script that runs too slowly
- **Code review:** review code and flag business risks

> **Note:** Many of these usages do not produce a single line of code but save hours. Explaining an inherited VBA model, or generating a comprehensive test plan, are among the most immediate wins.

© 2026 Necflow AI & David Schraub Actuarial Consultancy. All rights reserved.

CHAPTER 5

Life & Annuities

Life and annuities (individual life, annuities, retirement, health riders) is where Claude Code delivers the most immediate value. Models are complex, calculations are numerous, and regulatory requirements (risk-based capital, principle-based reserving under VM-20, US GAAP LDTI, asset adequacy testing) impose a high standard of rigor and documentation. This chapter follows the organization of a typical insurance company: ALM, Assets, Modeling, Risk Measurement, Model Validation, Valuation, Product, Investments, and Health.

5.1 Risk Division — ALM Team

The ALM team ensures assets and liabilities stay matched. It projects cash flows, manages strategic allocation and builds the asset-liability projection models.

Use case 1 — Valuing a bond portfolio

Situation: The ALM actuary must value a bond portfolio and project its cash flows (coupons, redemptions) with credit risk included. That covers fixed-rate, floating-rate and inflation-linked bonds, with default probabilities and recovery rates. These valuations feed reserve adequacy work, stress tests and allocation decisions.

What Claude Code will design: a library of valuation functions, one per bond type, that take the security's characteristics and market curves as inputs and return market value, book value and yield to maturity. Each function handles curve interpolation and credit risk.

The market value of a risky bond in a risk-neutral framework is written:

$$MV(t) = \sum_{i=1}^{T-t} \frac{Coupon(t+i)}{(1+r(t,i))^{i}} + \frac{Redemption(T)}{(1+r(t,T-t))^{T-t}}$$

where the coupon of a risky bond combines surviving and defaulted notionals:

$$Coupon(t) = CouponRate(t) \times [SurvivingNotional(t) + RecoveryRate \times DefaultedNotional(t) \times \mathbb{1}_{Recovery}]$$

The inputs, concretely

The prompt below manipulates four kinds of data. Knowing what each one represents is what makes the function signatures readable:

- **The zero-coupon curve (zc_curve):** the risk-free rate curve from your economic scenario generator, given as points (t, maturity, rate): at valuation time t, the annualized rate for each quoted maturity (1 to 40 years, typically). Intermediate maturities are interpolated — hence the first function.

© 2026 Necflow AI & David Schraub Actuarial Consultancy. All rights reserved.

- **Notional schedules (surviving_notional, defaulted_notional):** outputs of the credit model. At each future date they split the initial notional between the part still performing and the part in default, driven by the cumulative default probability curve.
- **Recovery (recovery_rate, recovery_flag):** the share of a defaulted notional that is recovered, and a flag saying whether defaulted notional keeps paying coupons weighted by that recovery rate.
- **Valuation parameters (spread, projection_basis):** the extra margin added to the curve when discounting this security, and the choice of risk-neutral or real-world discounting.

The companion pack of this use case ships a worked example of every one of these structures — a 15-bond portfolio, the curves and the default probabilities — with the column-by-column dictionary in its CLAUDE.md.

■ Claude Code Prompt

claude

```
> I want a Python bond valuation library for a life insurance ALM
> model. Create a module bond_valuation.py with:
>
> 1. A function interpolate_zc(t, maturity, zc_curve) that
>    interpolates a zero-coupon rate by cubic spline. The curve is
>    given as points (t, maturity, rate). Use flat extrapolation
>    beyond the last point.
>
> 2. A function mv_fixed_rate(t, maturity, coupon_freq, coupon_rate,
>    redemption_rate, surviving_notional, defaulted_notional,
>    recovery_rate, recovery_flag, zc_curve, default_prob_curve,
>    spread, projection_basis) that computes the market value of a
>    risky fixed-rate bond. The coupon combines the surviving
>    notional plus the defaulted notional weighted by the recovery
>    rate when coupon recovery is active. The default probability
>    curve drives surviving/defaulted notionals; the spread is added
>    to the rate curve; the projection basis (risk-neutral or
>    real-world) determines discounting.
>
> 3. A function mv_floating_rate(...) identical but where the coupon
>    rate is a forward rate rebuilt from the ZC curve.
>
> 4. A function mv_inflation_linked(...) with two curves (real and
>    nominal) and a floor on redemption: the redemption paid is the
>    max of the inflation-indexed notional and the initial notional.
>
> 5. A function ytm(...) that computes yield to maturity by
>    bisection: the single rate equating the discounted future cash
>    flows to the current book value.
>
> Write clear docstrings, handle coupon_freq=0 (zero-coupon), and
> add unit tests checking: MV=0 after maturity, consistency when
> default probabilities are zero (constant surviving notional), and
> YTM convergence.
```

Expected inputs:

- Each bond's characteristics (maturity, coupon frequency and rate, redemption rate)
- Surviving and defaulted notionals from the credit model, and the default probability curve
- Zero-coupon curves (from your economic scenario generator), recovery rate, coupon recovery flag

© 2026 Necflow AI & David Schraub Actuarial Consultancy. All rights reserved.

- The additional spread over the curve, and the projection basis (risk-neutral or real-world)

Expected outputs:

- A tested Python module with valuation functions per bond type
- Market value, book value and yield to maturity for each security
- A battery of unit tests validating the edge cases

Note: With credit risk switched off, default probabilities are zero: the surviving notional stays equal to the initial notional and the defaulted notional is nil. The valuation then collapses to classic discounted cash flows. Other simplifying assumptions are implied (e.g. no call option, no FX, no floating rate).

Companion resources: replicate this use case at home — the project folder (CLAUDE.md, ready-to-paste prompts, dataset and instructions) is available for download: Google Drive folder. The pack adds a 15-bond synthetic portfolio (four bond types, ratings AAA to BB), the zero-coupon and default curves, and reference values to check your run against.

Use case 2 — Asset-liability projection engine

Situation: The Risk division needs an asset-liability projection tool: a calculation engine that projects a life insurer's assets and liabilities simultaneously over 40+ years, time step by time step. In practice it starts from detailed functional specifications and assumption tables (portfolios, economic context, biometric and policyholder behavior assumptions). It is the most structuring project a modeling team owns.

What Claude Code will design: a modular projection engine, fed by assumption tables, able to project every asset class and every liability product while handling asset-liability interactions at each time step (credited rates, dividend policy, allocation rebalancing, asset sales to pay benefits).

Engine inputs (assumption tables):

- Asset portfolio: bonds (fixed, floating, inflation-linked), index assets (equities, real estate, private equity, funds), derivatives (caps, floors, swaps, puts, calls)
- Liability portfolio: deferred annuities, fixed and indexed annuities, universal life with secondary guarantees, term and whole life blocks
- Economic context: ESG scenarios (rate curves, equity returns, inflation, spreads), risk-neutral or real-world
- Biometric and behavioral assumptions: mortality tables, structural and dynamic lapse laws, annuitization and partial withdrawal rates, premium persistency

On the liability side, the engine projects: account values by fund, premiums and loads, benefits (deaths, full and partial surrenders, dynamic lapses, maturities and annuitizations), credited interest, guaranteed minimum rates, and the dividend / credited-rate policy.

On the asset side: bond cash flows (coupons, redemptions), index asset performance and dividends, real estate income, market and book values by bucket, realized and unrealized gains and losses, rebalancing toward the strategic allocation.

© 2026 Necflow AI & David Schraub Actuarial Consultancy. All rights reserved.

■ Claude Code Prompt (adapt to your specifications)

claude

```
> Context: I am building an asset-liability projection engine for a
> US life insurer, in Python. I will provide my functional
> specifications in the ./specs folder (an asset document, a
> liability document, and a document describing the year-end
> closing algorithms). Read them in full before starting.
>
> EXPECTED ARCHITECTURE:
> - A projection engine with a monthly loop over 480 steps (40
>   years).
> - An abstract Asset class with subclasses: FixedRateBond,
>   FloatingRateBond, InflationLinkedBond, Equity, RealEstate,
>   PrivateEquity, Fund, Cash, and Derivative (Cap, Floor, Swap,
>   Put, Call).
> - An abstract Liability class with subclasses: DeferredAnnuity,
>   FixedIndexedAnnuity, UniversalLife, TermBlock, PayoutAnnuity.
> - An Assumptions module that loads the tables from ./data (asset
>   portfolio, liability model points, ESG scenarios, biometric and
>   behavioral tables).
>
> TIME-STEP LOGIC (in this order):
> 1. Initialization / start of period: asset recalibration.
> 2. Liability roll-forward: age and duration updates, mortality and
>    lapse application (structural + dynamic), surviving counts.
> 3. Asset performance: coupons, dividends, revaluation, investment
>    income and realized gains/losses.
> 4. Benefits: full/partial surrenders, deaths, maturities. Sell
>    assets if cash is insufficient.
> 5. Premiums: initial, scheduled and flexible.
> 6. Credited interest: guaranteed minimum rate, then target
>    credited rate.
> 7. Credited-rate / dividend policy: apply the company's crediting
>    strategy (parametrizable), manage the interest maintenance
>    logic.
> 8. Investment strategy: rebalance the allocation toward the
>    strategic target (buys/sells by asset class).
> 9. Update account values and reserves by product.
>
> DYNAMIC LAPSE: model it as a function of the spread between the
> market reference rate and the credited rate (capped sigmoid),
> parametrizable through the behavioral assumptions table.
>
> OUTPUTS: at each step, export net income, the credited rate,
> account values, reserves, and the market value of the portfolio
> by class. Produce a results file in Parquet format (a compressed
> columnar data format) and a summary report.
>
> QUALITY: structure the code in clear modules, add unit tests on
> each brick (a single bond, a single contract), and global
> consistency tests (reserve conservation, asset-liability balance
> equality). Document every module.
>
> Start by proposing the project structure and the class
> signatures, then wait for my validation before writing the
> implementation.
```

Expected inputs:

© 2026 Necflow AI & David Schraub Actuarial Consultancy. All rights reserved.

- Functional specifications (asset and liability) in a dedicated folder
- Assumption tables: asset and liability portfolios, ESG scenarios, biometric and behavioral tables

Expected outputs:

- A modular, tested, documented projection engine
- Projected cash flows and reserves step by step, assets and liabilities
- A summary report and a usable results file

> **Tip:** Make Claude Code work in stages. Ask for the architecture and class signatures first, validate them, then let it implement module by module with tests along the way. That is the best path to a reliable engine on a project of this size.

> **Note:** Asset-liability projection models are highly complex tools. Their parameterization and business-specific features can be embedded in numerous components throughout the model. As such, the output generated from this prompt would most likely serve as a starting point for a cell-by-cell validation approach rather than a complete solution. Critical elements such as economic scenario calibration, accounting rules, complex asset projections, investment strategies, expense modeling, and numerous other actuarial assumptions would still require detailed expert review.

Use case 3 — ALM monitoring dashboard

> **Situation:** The ALM team regularly produces indicators for the risk committee: asset and liability duration, duration gap, coverage ratio, sensitivity to rate shocks, liquidity gap. These indicators are often scattered across several manually updated Excel files, then re-copied into decks. The goal is to centralize everything in an up-to-date interactive dashboard.

What Claude Code will design: a local web application (Streamlit) that reads the portfolio data, computes the ALM indicators and displays them in interactive charts, with filters and an export option.

■ Claude Code Prompt

claude

```
> Create a Streamlit ALM monitoring dashboard for a life insurer.
> It reads data from ./data (asset positions in Excel, liability
> cash flow schedule in CSV) and displays:
> - At the top: 4 key indicators (asset duration, liability
>   duration, duration gap, coverage ratio) as cards.
> - A chart of the projected cash flow gap over 40 years (asset
>   flows minus liability flows by year).
> - A sensitivity curve of the coverage ratio to rate shocks, from
>   -150 bp to +150 bp in 25 bp steps.
> - A pie chart of portfolio composition by asset class.
> - A filterable table of positions, with Excel export.
> Use a sober theme and professional colors. Add a data refresh
> button and handle missing files cleanly.
```

Expected inputs:

- Asset positions (Excel), liability cash flow schedule (CSV)

© 2026 Necflow AI & David Schraub Actuarial Consultancy. All rights reserved.

Expected outputs:

- A Streamlit app launched with one command, with indicators, charts and export

> **Note:** If you do not specify one aspect, the tool will provide a solution internally in order to move forward. Macaulay duration, modified duration, effective duration, key rate duration... Parallel shocks, steepening shocks, PCA shocks... Claude will pick and move forward, and may tell you what was picked in the documentation it produces.

Use case 4 — Strategic asset allocation

> **Situation:** The insurer must set the target allocation of its portfolio (bonds, equities, real estate, cash) under multiple constraints: a target yield to support credited rates, RBC capital charges by asset class, liquidity constraints for surrenders, a target duration to match liabilities. It is a constrained optimization problem traditionally handled in Excel Solver, with its limits.

What Claude Code will design: an allocation optimizer that builds the efficient frontier under regulatory and business constraints and proposes the optimal risk-adjusted allocation.

■ Claude Code Prompt

claude

```
> Build a strategic allocation optimizer in Python (scipy.optimize).
> Asset classes: government bonds, corporate bonds, US equities,
> international equities, real estate, cash.
> Inputs (Excel file): expected return and volatility by class,
> correlation matrix, duration and NAIC RBC C-1 charge by class.
> Constraints: minimum portfolio yield 2.5%; asset duration between
> 8 and 12 years; total C-1 capital charge below 15% of surplus;
> equity bucket capped at 20%; cash bucket at least 5% for
> liquidity.
> Objective: maximize the Sharpe ratio (risk-adjusted return).
> Outputs: the optimal allocation, the efficient frontier plot, an
> allocation table with the binding constraints identified, and a
> sensitivity analysis of the allocation to the target yield.
```

Expected outputs:

- The optimal target allocation and the efficient frontier
- The detail of binding constraints and a sensitivity analysis

> **Note:** The liability side is assumed to have been shared through a different mechanism. Numerous elements are not specified here (e.g. volatility, convexity, tail risk, rating agency capital), and Claude Code will fill the gap with its own assumptions.

5.2 Risk Division — Assets and Investment Strategy

This section covers index asset projection beyond the plain vanilla non-callable bond, consolidated portfolio valuation and the strategic rebalancing logic.

© 2026 Necflow AI & David Schraub Actuarial Consultancy. All rights reserved.

Use case 5 — Line-by-line equity projection (performance and dividends)

Situation: ALM and investment teams must project each equity line individually, not in aggregate: unrealized and realized gains and losses, dividends paid, market value. Unlike bonds, there are no deterministic flows: performance depends entirely on the ESG market scenarios.

What Claude Code will design: a module that projects, CUSIP by CUSIP, the market value of each equity holding under the ESG return scenarios, separating total return performance, distributed dividends and gains/losses.

The market value of an index asset evolves with the scenario's total return performance, net of the distributed dividend yield:

$$MV(t) = MV(t-1) \times (1 + Perf_{TR}(t)) - Dividends(t)$$

■ Claude Code Prompt

claude

```
> Create a Python module equity_projection.py to project an equity
> portfolio line by line in an ALM model.
> Inputs: an equity portfolio (identifier, initial market value,
> initial book value, dividend yield) and ESG total return
> scenarios by time step.
> For each line and each time step, compute:
> - the scenario total return applied to the line,
> - distributed dividends (dividend yield applied to MV),
> - the new market value = previous MV x (1 + TR perf) - dividends,
> - unrealized gains/losses = MV - book value,
> - realized gains/losses in case of sale.
> Handle multiple stochastic scenarios in parallel (vectorize with
> numpy).
> Outputs: one DataFrame per scenario with line-level detail by
> step, plus a portfolio aggregate. Add consistency tests (sum of
> line MVs = portfolio MV).
```

Expected inputs:

- Equity portfolio (initial MV and book value, dividend yield by line)
- ESG total return scenarios

Expected outputs:

- Line-by-line projection: MV, dividends, unrealized and realized gains/losses
- An aggregated portfolio view consistent with the line-level detail

Note: As with any generated module, validate the output against a small hand-built benchmark (a few positions reproduced in a spreadsheet) before relying on it.

© 2026 Necflow AI & David Schraub Actuarial Consultancy. All rights reserved.

Use case 6 — Consolidated multi-class portfolio valuation

> **Situation:** The insurer holds a portfolio of bonds, equities, real estate, funds and cash. A consolidated view is needed for reporting, ALM monitoring and capital calculations. Today that consolidation is often a manual assembly of several files — error-prone and slow.

What Claude Code will design: a consolidation tool that aggregates the valuation of all asset classes, computes global indicators (duration, yield, breakdown by rating, by maturity, by class) and produces a summary report.

■ Claude Code Prompt

claude

```
> Create a Python tool for consolidated valuation of an insurer's
> asset portfolio. It reads several sources in ./data: bonds
> (Excel), equities (Excel), real estate (CSV), funds (CSV), cash
> (CSV). It produces:
> - a consolidated valuation (total market and book value, and by
>   asset class),
> - the portfolio duration (weighted average of bond durations),
> - the average portfolio yield,
> - the breakdown by asset class, by credit rating, by maturity
>   bucket,
> - a consistency check (sum of classes equals the total).
> Output: a consolidated Excel report with a summary sheet and one
> detail sheet per class, plus charts (breakdown, maturity
> profile). Handle missing files cleanly.
```

Expected inputs:

- Position files by asset class

Expected outputs:

- A consolidated Excel report with summary, detail and charts
- Global indicators: duration, yield, breakdowns

Use case 7 — Investment strategy and allocation rebalancing

> **Situation:** At each projection step, the insurer rebalances its asset allocation toward the strategic target. That means buying and selling securities under precise rules: which classes to reinforce, which securities to sell, and for bond purchases, which maturities and coupons to pick. It is a transversal, time-consuming topic for modeling teams.

What Claude Code will design: a rebalancing module that, at each time step, computes the gaps between the current and target allocations, derives the buy and sell amounts by class, and constructs the new bond holdings (maturity and coupon chosen per the investment policy).

© 2026 Necflow AI & David Schraub Actuarial Consultancy. All rights reserved.

■ Claude Code Prompt

```
claude

> Create a module investment_strategy.py for an ALM projection
> engine. At each time step it applies the reallocation strategy:
>
> 1. Compute the current allocation by class (from market values)
>    and compare it to the strategic target provided as an
>    assumption.
> 2. Determine the amounts to buy or sell by class to return to
>    target, respecting a tolerance corridor (e.g. only reallocate
>    if the gap exceeds 2%).
> 3. For sales: apply a rule (e.g. first in first out, or pro rata)
>    and compute realized gains/losses.
> 4. For bond purchases: build new securities per the investment
>    policy (target maturity, coupon at the current market rate for
>    the chosen maturity, target credit rating).
> 5. Update the portfolio and return the transaction detail.
>
> Inputs: current portfolio, target allocation, tolerance corridor,
> investment policy (target maturities and ratings), market curves.
> Outputs: rebalanced portfolio, transaction log (buys/sells by
> class), realized gains/losses.
> Add tests: after rebalancing, the allocation is inside the
> corridor; total value is conserved (before costs).
```

Expected inputs:

- Current portfolio, target allocation, tolerance corridor, investment policy, market curves

Expected outputs:

- Rebalanced portfolio, transaction log by class, realized gains and losses

5.3 Risk Division — Modeling Team

The modeling team, usually inside the Risk division, designs and maintains the models used by every other team. It writes new code, maintains legacy VBA, and documents the models.

Use case 8 — Developing VBA valuation functions

> **Situation:** Many production projection models still run on Excel/VBA. The team must develop specialized functions (interpolation, valuation, forward rates) from mathematical specifications. Claude Code dramatically speeds up this work by generating robust, documented VBA.

Here are the main functions developed with Claude Code for a bond projection model, with an excerpt of the generated code.

ZcInterpolCubic — cubic interpolation of rate curves

Interpolates a zero-coupon rate for any maturity, by cubic spline, from the curve provided by the ESG.

© 2026 Necflow AI & David Schraub Actuarial Consultancy. All rights reserved.

■ Claude Code Prompt

```
claude

> Create a VBA function ZcInterpolCubic(time, maturity, zc_table)
> that interpolates a zero-coupon rate by constrained cubic spline
> from a ZC rate table. It takes the time step, the maturity in
> months, and the Excel range of the curve. Build a reusable spline
> object and return the interpolated rate. Use flat extrapolation
> outside the bounds.
```

Excerpt of the generated function:

```
VBA

Function ZcInterpolCubic(ByVal time As Double, _
                         ByVal maturity As Double, _
                         ByRef zc_table As Range) As Double
    Dim n As Long, i As Long
    n = zc_table.Rows.Count
    Dim xs() As Double, ys() As Double
    ReDim xs(1 To n): ReDim ys(1 To n)
    For i = 1 To n
        xs(i) = zc_table.Cells(i, 1).Value   ' maturities
        ys(i) = zc_table.Cells(i, 2).Value   ' ZC rates
    Next i
    ' Flat extrapolation outside bounds
    If maturity <= xs(1) Then ZcInterpolCubic = ys(1): Exit Function
    If maturity >= xs(n) Then ZcInterpolCubic = ys(n): Exit Function
    ' Natural cubic spline interpolation
    ZcInterpolCubic = CubicSpline(xs, ys, maturity)
End Function
```

MV_Float_Det — market value of a floating-rate bond

Computes the deterministic market value of a floating-rate bond. The function loops over future steps, checks whether a coupon falls due (per the frequency), retrieves surviving and defaulted notionals, interpolates the ZC rate and discounts the flows.

■ Claude Code Prompt

```
claude

> Create a VBA function MV_Float_Det that computes the market value
> of a floating-rate bond deterministically. Inputs: time step t,
> total maturity, coupon frequency, redemption rate, recovery rate,
> coupon recovery flag, columns of surviving/defaulted notionals
> and coupon rates, nominal ZC curves, default probability table.
> Use ZcInterpolCubic for interpolation. The coupon combines the
> surviving notional plus the defaulted notional weighted by the
> recovery rate when the flag is active. Handle freq=0
> (zero-coupon).
```

Excerpt of the generated function:

```
VBA
```


© 2026 Necflow AI & David Schraub Actuarial Consultancy. All rights reserved.

```
Function MV_Float_Det(ByVal t As Long, ByVal mat As Long, _
                      ByVal freq As Double, ByVal redemp As Double, _
                      ByVal recov As Double, ByVal recovCpn As Boolean, _
                      ByRef nomSurv As Range, ByRef nomDef As Range, _
                      ByRef cpnRate As Range, ByRef zcCurve As Range) As Double
    Dim i As Long, coupons As Double, df As Double, vm As Double
    Dim cpnSurv As Double, cpnDef As Double, activRecov As Double
    activRecov = IIf(recovCpn, 1#, 0#)
    coupons = 0#
    For i = t + 1 To mat
        If (i - t) Mod (12 / freq) = 0 Then      ' a coupon falls due
            df = 1# / (1# + ZcInterpolCubic(t, i - t, zcCurve)) ^ ((i - t) / 12)
            cpnSurv = nomSurv.Cells(i, 1).Value * cpnRate.Cells(i, 1).Value / freq
            cpnDef = nomDef.Cells(i, 1).Value * cpnRate.Cells(i, 1).Value / freq
            coupons = coupons + df * (cpnSurv + activRecov * recov * cpnDef)
        End If
    Next i
    df = 1# / (1# + ZcInterpolCubic(t, mat - t, zcCurve)) ^ ((mat - t) / 12)
    vm = coupons + df * redemp * (nomSurv.Cells(mat, 1).Value _
         + recov * nomDef.Cells(mat, 1).Value)
    MV_Float_Det = vm
End Function
```

YTM_Float_Det — yield to maturity

Computes a floating-rate bond's yield to maturity by bisection. The YTM is the single rate that, applied to all future flows, equates their discounted sum to the previous step's book value. The search runs between wide bounds with a fine tolerance.

■ Claude Code Prompt

claude

```
> Create a VBA function YTM_Float_Det that computes yield to
> maturity by bisection. The YTM equates the discounted sum of
> future flows to the previous book value (bv_prev). Same inputs as
> MV_Float_Det plus bv_prev. Bisection between -80% and +2000%, 100
> iterations max, tolerance 1e-8. Handle freq=0.
```

Excerpt of the generated function:

VBA

```
Function YTM_Float_Det(ByVal bv_prev As Double, ByVal t As Long, _
                       ByVal mat As Long, ByVal freq As Double, _
                       ByRef flows As Range) As Double
    Dim lo As Double, hi As Double, mid As Double, val As Double
    Dim iter As Long
    lo = -0.8: hi = 20#            ' search bounds
    For iter = 1 To 100
        mid = (lo + hi) / 2#
        val = SumDiscountedFlows(flows, t, mat, mid) - bv_prev
        If Abs(val) < 0.00000001 Then Exit For   ' tolerance reached
        If val > 0 Then lo = mid Else hi = mid   ' bisection
    Next iter
    YTM_Float_Det = mid
End Function
```


© 2026 Necflow AI & David Schraub Actuarial Consultancy. All rights reserved.

Other functions in the same module (CMS rates, forward CMS, forward floating rates, inflation-linked bond valuation) were generated with the same approach: provide the mathematical specification, Claude Code produces tested VBA.

> **Tip:** For VBA, always provide the complete mathematical specification (formula, variables, conventions) rather than a vague description. The more precise the spec, the more correct the code on first generation.

> **Companion resources:** replicate this use case at home — the project folder (CLAUDE.md, ready-to-paste prompts, dataset and instructions) is available for download: Google Drive folder. The pack projects the market value of a fixed-rate bond over 480 monthly steps from a projected risk-free curve, with two exercises: a VBA function in Excel, and a small Streamlit app — a preview of the internal tools of chapter 8.

Use case 9 — Modeling dynamic lapses

> **Situation:** Dynamic lapse behavior is sometimes coded depending on the spread difference between the rate credited on the contract and a market reference rate. When market rates rise and the crediting rate does not follow, surrenders accelerate. It sits at the heart of reserve adequacy and cash flow testing, and it is hard to calibrate. Actuaries spend a lot of time on it.

What Claude Code will design: a module implementing the dynamic lapse law (as a function of the credited-vs-market spread), calibrating it on observed history, and backtesting the calibration.

The dynamic lapse rate is typically modeled as a capped function of the gap between the rate policyholders expect and the credited rate:

$$DL(t) = \min\left(DL_{max},\ \max\left(DL_{min},\ f(CompetitorRate(t) - CreditedRate(t))\right)\right)$$

■ Claude Code Prompt

```
claude

> Create a Python module dynamic_lapse.py that:
> 1. Implements a parametric dynamic lapse law: the dynamic lapse
>    rate is a capped function (sigmoid or regulatory-style step
>    function) of the spread between the policyholder's expected
>    rate and the credited rate. Parameters: floor, cap,
>    thresholds, slope. Provide both forms (step and sigmoid).
> 2. Calibrates the parameters on observed history (CSV file with
>    rate spread and observed lapse rate) by least squares.
> 3. Allows backtesting: compare predicted vs observed lapses on a
>    holdout period, with error metrics and a predicted-vs-actual
>    chart.
> Outputs: calibrated parameters, plots of the calibrated law and
> the backtest, a calibration quality report.
```

Expected inputs:

- Observed history (rate spread, observed lapse rate)

© 2026 Necflow AI & David Schraub Actuarial Consultancy. All rights reserved.

Expected outputs:

- Calibrated dynamic lapse law, charts, backtesting report

Use case 10 — Stochastic reserves — scenario pipeline and CTE

> **Situation:** Under principle-based reserving (VM-20), the stochastic reserve is computed from liability cash flows projected over a full set of economic scenarios, discounted, and summarized by a conditional tail expectation (CTE 70). It is the central calculation of modern US life valuation.

What Claude Code will design: a pipeline that reads projected liability cash flows across all stochastic scenarios, discounts them along each path, computes the scenario reserve distribution and the CTE metric, with convergence diagnostics.

$$Reserve_{CTE70} = \text{mean } of\ the\ worst\ 30\%\ of\ \left\{ PV_s\left(\sum_t CF_t\right) \right\}_{s=1..N}$$

■ Claude Code Prompt

claude

```
> Build a Python stochastic reserve pipeline for a US life insurer.
> Inputs: projected liability cash flows (parquet file, dimensions
> scenario x time step x flow type: benefits, premiums, expenses),
> and the path-wise discount rates from the scenario set.
> The pipeline must:
> 1. Discount each scenario's net cash flows along its own path.
> 2. Build the distribution of scenario present values (GPVAD-style
>    greatest PV of accumulated deficiencies if provided, else PV
>    of net flows).
> 3. Compute CTE 65, CTE 70 and CTE 90 of the distribution, and the
>    corresponding stochastic reserve.
> 4. Produce a breakdown of the reserve by flow type and by
>    projection year.
> 5. Check Monte Carlo convergence (CTE vs number of scenarios) and
>    flag if the scenario count is insufficient.
> Outputs: the CTE metrics, breakdown tables, convergence charts,
> and an Excel export. Add consistency checks (components sum to
> the total).
```

Expected inputs:

- Projected liability cash flows (scenarios x time steps x flow type), path-wise discount rates

Expected outputs:

- CTE 65/70/90 metrics and the stochastic reserve, breakdowns and convergence diagnostics

© 2026 Necflow AI & David Schraub Actuarial Consultancy. All rights reserved.

Use case 11 — Modeling a GMDB floor guarantee

Situation: The guaranteed minimum death benefit on variable annuities promises a floor (often total premiums paid) even when the account value has fallen. The risk is asymmetric: the cost depends on the gap between the account value and the guaranteed floor. This calculation is usually coded directly in the projection software; a standalone replica remains useful for validation, sensitivity testing and training.

What Claude Code will design: a tool that projects the guarantee cost under market scenarios: for each scenario and time step, it compares the account value to the guaranteed floor and computes the expected cost weighted by the probability of death.

$$Cost_{GMDB}(t) = q_x(t) \times \max(0,\ Guarantee(t) - AV(t))$$

Claude Code Prompt

claude

```
> Create a Python module gmdb_guarantee.py that projects the cost
> of a guaranteed minimum death benefit on variable annuity
> contracts.
> Inputs: initial account value, guaranteed floor (e.g. total
> premiums paid), mortality table (death rates by age), and ESG
> scenarios of separate account returns.
> For each scenario and each time step:
> - project the account value with the scenario return,
> - compute the guarantee cost at death = max(0, floor - account
>   value),
> - weight by the probability of death over the period,
> - accumulate and discount the cost over the horizon.
> The guarantee cost is the discounted average across scenarios.
> Outputs: expected guarantee cost, its distribution across
> scenarios, a cost-by-year chart, and a sensitivity analysis to
> the floor level and to fund volatility.
```

Expected inputs:

- Initial account value, guaranteed floor, mortality table, fund return scenarios

Expected outputs:

- Expected guarantee cost, distribution, cost by year, sensitivities

Use case 12 — Automatic technical documentation of models

Situation: Every actuarial model must ship with technical documentation: specifications, algorithm descriptions, assumptions, formulas, limitations. Auditors, state examiners and internal validation (per ASOP 41 and ASOP 56) require it. Yet documentation usually lags the code, incomplete or stale. It is one of the great year-end pains.

What Claude Code will design: complete technical documentation generated from the model's code, in Markdown, including LaTeX formulas and parameter descriptions.

© 2026 Necflow AI & David Schraub Actuarial Consultancy. All rights reserved.

■ Claude Code Prompt

```
claude

> Read the full code of the reserving model in ./models and
> generate the complete technical documentation in Markdown. For
> each module and each function:
> - a description of its business role,
> - input and output parameters with types and units,
> - the mathematical formulas used, written in LaTeX,
> - implicit assumptions and limitations,
> - dependencies on other modules.
> Structure the document with a table of contents, one section per
> module, and a transverse section describing the calculation
> chain. Write in a style suited to a technical note intended for
> auditors and internal model validation.
```

Expected inputs:

- The model's code folder

Expected outputs:

- Complete Markdown technical documentation, LaTeX formulas included, ready for review

5.4 Risk Division — Risk Measurement and Capital

The risk measurement team quantifies risks and computes capital requirements.

Use case 13 — NAIC risk-based capital calculator

> **Situation:** The RBC calculation is heavy and recurring: compute each risk component (C-0 affiliates, C-1 asset risk, C-2 insurance risk, C-3 interest rate and market risk, C-4 business risk), then combine them through the covariance formula. Teams spend days checking large Excel files, with high error risk.

What Claude Code will design: an RBC calculator that takes the exposures and factors by component, applies the covariance formula, and produces the authorized control level RBC and the RBC ratio with a visual decomposition.

$$RBC_{after\ cov} = C_0 + C_{4a} + \sqrt{(C_{1o} + C_{3a})^2 + C_{1cs}^2 + C_2^2 + C_{3b}^2 + C_{4b}^2}$$

© 2026 Necflow AI & David Schraub Actuarial Consultancy. All rights reserved.

■ Claude Code Prompt

```
claude

> Build a life RBC calculator in Python.
> Input: an Excel file with exposures and factors by component:
> C-0 (affiliates), C-1o (bonds by NAIC class, mortgages, other
> invested assets), C-1cs (common stock), C-2 (net amount at risk
> by product), C-3a (interest rate risk), C-3b (health credit
> risk), C-4a and C-4b (business risk).
> The calculator must:
> 1. Compute each component's requirement (exposure x factor,
>    with size adjustments where applicable).
> 2. Combine them with the covariance formula: C0 + C4a +
>    sqrt((C1o+C3a)^2 + C1cs^2 + C2^2 + C3b^2 + C4b^2).
> 3. Derive authorized control level RBC and the RBC ratio (total
>    adjusted capital / ACL).
> 4. Flag the regulatory action levels (company action, regulatory
>    action, authorized control, mandatory control).
> Outputs: RBC by component, the covariance benefit, a waterfall
> chart of the decomposition, and an Excel export with the factor
> tables hard-coded and commented with their source.
```

Expected inputs:

- Exposures by component and total adjusted capital

Expected outputs:

- RBC by component, ACL RBC, RBC ratio, action-level flags, waterfall chart, Excel export

> **Note:** The NAIC RBC spreadsheet does that as well. The value here lies in automation and integration with your own data pipeline.

> **Companion resources:** replicate this use case at home — the project folder (CLAUDE.md, ready-to-paste prompts, dataset and instructions) is available for download: Google Drive folder. The pack's synthetic exposure file comes with a hand-checked reference table (RBC by component, covariance benefit, ACL and ratio), so you can verify every figure of your run.

Use case 14 — Automating ORSA stress tests

> **Situation:** Under ORSA, the insurer projects its solvency over 3 to 5 years under several stress scenarios: rate shock, equity crash, mortality deterioration, mass lapse, combinations. Each scenario requires a full balance sheet projection. Automation is essential to meet the filing calendar.

What Claude Code will design: an orchestrator that, from the initial balance sheet and a stress catalog, projects the capital ratio over the ORSA horizon for each scenario and produces a comparative report.

© 2026 Necflow AI & David Schraub Actuarial Consultancy. All rights reserved.

■ Claude Code Prompt

```
claude

> Create a Python ORSA stress test orchestrator. It takes the
> initial balance sheet (capital, RBC, reserves) and a stress
> scenario catalog defined in a YAML configuration file:
> - baseline scenario,
> - rates -100 bp,
> - equities -40%,
> - lapses x2,
> - combined rates + equities,
> - a deteriorated mortality scenario.
> For each scenario, project over 5 years: available capital, the
> RBC requirement, and the RBC ratio. Use parametrizable
> projection functions (I will provide the business logic, or
> start from simple documented assumptions).
> Outputs: a table of RBC ratios by scenario and by year, a chart
> of ratio trajectories, identification of scenarios where the
> ratio falls below the company action level, and a PDF summary
> report. The scenario catalog must be easily extensible.
```

Expected inputs:

- Initial balance sheet, stress scenario catalog (configuration file)

Expected outputs:

- Capital ratio trajectories by scenario, breach alerts, summary report

> **Note:** As mentioned earlier, whatever is not specified will be filled out by Claude Code. For example, how the 5-year projections are performed is not spelled out in the snippet shared.

5.5 Model Validation and Actuarial Oversight

Model validation — whether housed in a dedicated team, with the appointed actuary, or under an ASOP 56 model governance framework — gives an independent opinion on models, methodologies and results. It is a control and second-opinion role.

Use case 15 — Validation tool by independent model replication

> **Situation:** To validate a calculation, a powerful approach is to replicate the model independently from its specification, then compare the replica's results to the official model's. Every gap flags either a bug or an ambiguous specification. Done by hand it takes weeks — it is exactly the kind of task where Claude Code excels.

What Claude Code will design: a validation tool that implements an independent version of the model from the functional specification, runs both versions on the same test data, and produces a detailed gap report.

© 2026 Necflow AI & David Schraub Actuarial Consultancy. All rights reserved.

■ Claude Code Prompt

```
claude

> I am performing model validation and I must review a reserve
> calculation feature. The functional specification is in
> ./specs/reserve_spec.docx and the official model results are in
> ./results/official_model.parquet.
> 1. Read the specification and implement, fully independently, a
>    reference version of the calculation in Python (do not look
>    at the official model's code, only the specification).
> 2. Run your reference version on the same model points as the
>    official model.
> 3. Compare results line by line: reserve per contract, per
>    product, and in total. Compute absolute and relative gaps.
> 4. Produce a validation report: list of gaps above a threshold,
>    analysis of likely causes (rounding, divergent assumption,
>    unhandled edge case), and a conclusion on compliance.
> Clearly flag any ambiguous points in the specification that
> forced you to make an assumption.
```

Expected inputs:

- The functional specification and the official model's results

Expected outputs:

- An independent replication of the model, a line-by-line gap report
- The list of specification ambiguities detected

> **Note:** This dual-implementation approach is one of the strongest ways to validate a model: two independent implementations that converge give high assurance; when they diverge, the gap points precisely to the problem.

Use case 16 — Critical review of a methodology memo

> **Situation:** Validation re-reads the methodology memos produced by modeling teams in order to opine. That critical review — hunting for unjustified assumptions, inconsistencies and unhandled cases — is time-consuming and demands sustained attention.

What Claude Code will design: a structured critical review of a methodology memo, identifying strengths, weaknesses, assumptions to challenge and questions to put to the team.

© 2026 Necflow AI & David Schraub Actuarial Consultancy. All rights reserved.

■ Claude Code Prompt

```
claude

> Act as a model validation actuary reviewing a methodology memo.
> The memo is in ./notes/methodology_memo.docx.
> Produce a structured critical review:
> - a summary of the proposed methodology in a few lines,
> - the list of key assumptions and an assessment of their
>   justification,
> - internal inconsistencies or contradictions spotted,
> - edge cases or situations not handled by the methodology,
> - potential non-compliance with VM-20,
> - a list of precise questions for the modeling team.
> Be rigorous and factual: cite the passages concerned. Do not
> paraphrase the memo — challenge it.
```

Expected outputs:

- A structured critical review: assumptions, inconsistencies, unhandled cases, questions for the team

5.6 Valuation and Financial Reporting

The valuation actuary computes reserves at every close and explains their movements.

Use case 17 — Statutory reserve calculation at the close

> **Situation:** At each close, the valuation actuary computes statutory reserves (net premium reserves, deficiency reserves, incidental reserves...) by running through the entire portfolio contract by contract.

What Claude Code will design: a reserve calculator that reads the portfolio, computes reserves per contract, aggregates them, and reconciles against the prior inventory.

■ Claude Code Prompt

```
claude

> Build a statutory reserve calculator in Python. It reads a
> portfolio of 500,000 contracts (parquet file). For each
> contract, compute the prospective net premium reserve with the
> prescribed mortality table (2017 CSO) and the contract's
> statutory valuation rate. Then compute, at portfolio level, the
> deficiency reserve where gross premiums are below net premiums,
> and the aggregate incidental reserves.
> Outputs: reserves aggregated by product, a contract-level detail
> file, and a reconciliation with the prior quarter's inventory
> (movement explained: new business effect, terminations effect,
> aging/interest effect).
> Optimize to process 500k contracts in reasonable time
> (vectorization).
```

Expected inputs:

- Contract portfolio, prescribed tables, prior inventory

Expected outputs:

- Reserves by product, contract-level detail, reconciliation with explained movement

© 2026 Necflow AI & David Schraub Actuarial Consultancy. All rights reserved.

> **Note:** Prospective net premium reserve and other reserve calculation routines are assumed to have been articulated in a different area of the project.

Use case 18 — Movement analysis and close controls

> **Situation:** At every close, the valuation actuary must explain the reserve movement between two valuations (roll-forward) and run a battery of consistency checks before locking the numbers. These controls are often manual and repeated identically every quarter.

What Claude Code will design: a tool that decomposes the reserve movement into effects (new business, benefits, interest, assumption changes) and automatically runs the close controls.

■ Claude Code Prompt

claude

```
> Create a Python roll-forward and close control tool.
> Inputs: the prior quarter and current quarter inventories (two
> parquet files, contract-level).
> 1. Decompose the reserve movement between the two closes into
>    effects: new business, terminations (lapses/deaths/
>    maturities), interest accretion, assumption changes, model
>    effect. The effects must sum back to the total movement.
> 2. Run consistency checks: no negative reserves, no duplicate
>    contracts, consistency between contract counts and flows,
>    movement by product inside plausible bounds.
> 3. Produce a close report with the roll-forward table, the list
>    of detected anomalies, and a green/red flag per control.
```

Expected outputs:

- A decomposed movement analysis, automated close controls, a summary report

5.7 Product Development

The product team designs and manages savings, retirement and protection products. It assesses profitability before launch and tracks performance over time.

Use case 19 — New product profitability analysis

> **Situation:** Before launching a savings or protection product, the product team must assess its profitability: project cash flows over the life of the contract and compute the present value of profits, the internal rate of return on capital, and the sensitivity of those metrics to key assumptions.

What Claude Code will design: a pricing tool that projects a product's cash flows and computes its profitability metrics, with sensitivity analysis.

© 2026 Necflow AI & David Schraub Actuarial Consultancy. All rights reserved.

■ Claude Code Prompt

```
claude

> Create a Python pricing tool for a fixed deferred annuity.
> Inputs: mortality table, credited rate strategy, expenses and
> loads, lapse assumptions, investment yield assumption, average
> premium.
> Project the cash flows over the full contract horizon and
> compute:
> - the value of new business (present value of future profits),
> - the profit margin (PVFP / PV of premiums),
> - the IRR on required capital and the break-even year,
> - the initial capital strain and its cost.
> Add a sensitivity analysis: lapses +/- 20%, yield +/- 50 bp,
> expenses +/- 10%. Produce a summary table and charts (profit
> profile over time, tornado chart of sensitivities).
```

Expected outputs:

- New business value, margin, IRR, break-even, sensitivity analysis

> **Note:** As mentioned above, Claude Code will fill out the necessary information not provided. Here, the economic scenario under which the cash flows shall be projected is not articulated: a likely level scenario would be used.

5.8 Corporate Finance and Investments

The finance division tracks investment portfolio performance, produces financial reporting and polices investment limits.

Use case 20 — Performance reporting and attribution

> **Situation:** The finance division produces recurring reports: portfolio performance, attribution by asset class, realized and unrealized gains tracking, investment limit compliance. These reports are often assembled by hand from several sources — slow and error-prone.

What Claude Code will design: a financial reporting generator that computes performance and attribution, checks limits and produces a formatted report.

© 2026 Necflow AI & David Schraub Actuarial Consultancy. All rights reserved.

■ Claude Code Prompt

```
claude

> Create a monthly financial reporting generator in Python. It
> reads portfolio positions (Excel) and market prices (CSV) and
> computes:
> - overall and per-asset-class performance for the month and
>   year-to-date,
> - performance attribution (each class's contribution to total
>   performance),
> - realized and unrealized gains and losses,
> - investment limit compliance (by class, by issuer, by rating)
>   with alerts on breaches.
> Produce a PDF report with charts: cumulative performance,
> portfolio breakdown, top and bottom positions.
```

Expected outputs:

- Performance and attribution by class, gains/losses, limit checks, PDF report

5.9 Health and Group Benefits

In health and group benefits, the actuary prices coverages (medical, dental, disability, group life), monitors experience and reserves disabled-life claims. The problems mix frequency, duration and severity.

Use case 21 — Health pricing and loss ratio monitoring

> **Situation:** In individual health, the actuary prices coverages from observed medical consumption and tracks the loss ratio by population. Pricing must reflect family composition, age, plan design, and trend by service category.

What Claude Code will design: a health pricing model by service category and a loss ratio monitoring dashboard by population.

■ Claude Code Prompt

```
claude

> Build an individual health pricing and monitoring model.
> Inputs: medical claims per member by service category
> (inpatient, outpatient, professional, pharmacy), with age,
> family status and plan design.
> 1. Compute average claim cost by category and age band.
> 2. Build a net premium by profile (age, family composition) by
>    aggregating categories, then apply loads and the target
>    margin.
> 3. Track the loss ratio by population and by category, and
>    detect categories drifting (12-month trend).
> Produce a monitoring dashboard and a pricing report.
```

Expected outputs:

- Net premium by profile and category, loss ratio dashboard, drift detection

© 2026 Necflow AI & David Schraub Actuarial Consultancy. All rights reserved.

> **Note:** As mentioned above, Claude Code will fill out the necessary information not provided. Portfolio composition trends and other elements are not included here.

Use case 22 — Disabled-life reserves for disability claims

> **Situation:** In group disability, the insurer pays benefits for as long as the claimant remains disabled. Reserving relies on claim termination rates (continuance tables) from prescribed tables or company experience. It is a specific calculation many actuaries meet without a dedicated tool.

What Claude Code will design: a disabled-life reserve calculator built on continuance tables, with projected benefit flows.

■ Claude Code Prompt

claude

```
> Create a Python disabled-life reserve calculator for group
> long-term disability. Inputs: the portfolio of open claims (age
> at disability, duration since disability, monthly benefit,
> benefit period, elimination period) and the continuance tables
> (probability of remaining disabled by age and duration). Use
> the statutory interest rate for discounting.
> For each claim, project future benefit payments weighted by the
> continuance probabilities, then discount to get the reserve.
> Handle transitions (recovery, death, retirement age limit) and
> benefit offsets if provided.
> Outputs: reserve per claim and aggregated, projected benefit
> flows, and a sensitivity analysis to the termination rates.
```

Expected inputs:

- Open claims portfolio, continuance tables, statutory interest rate

Expected outputs:

- Reserve per claim and in aggregate, projected flows, sensitivity to termination rates

Use case 23 — Pandemic experience monitoring — data-visualization storytelling

> **Situation:** A data-visualization challenge run by the Institut des Actuaires, the French actuarial society, during the COVID-19 epidemic: make public hospital data speak — admissions, ICU, vaccination, economic impact — and tell a coherent story to management and non-specialist colleagues. Mortality data is deliberately out of scope. The exercise captures a real part of the job: pandemic experience monitoring and executive communication.

What Claude Code will design: an end-to-end analysis of three public datasets (hospital, vaccination, economic), the exploratory work, a set of publication-ready visualizations with a clear message per chart, and a 10-15 page PDF deck that follows a narrative thread: the arrival of the pandemic, the successive waves, regional disparities, the effect of vaccination on hospitalizations, the economic cost, and the lessons for pandemic risk management.

© 2026 Necflow AI & David Schraub Actuarial Consultancy. All rights reserved.

■ Claude Code Prompt

```
claude

> Read the CLAUDE.md file at the root of the project. You are an
> actuary's assistant, given a data visualization challenge on
> COVID-19 hospital data (folders 1_hospitalieres, 2_vaccins,
> 3_economiques in data/). Run through all the steps without
> stopping:
> 1. Explore the 3 folders, identify files, columns and periods
> 2. Load and clean the data
> 3. Build a complete actuarial analysis with a clear narrative:
>    the pandemic's arrival and successive waves, geographic
>    disparities, the effect of vaccination on hospitalizations
>    and ICU, the economic impact, and the lessons for pandemic
>    risk management
> 4. Produce the visualizations (high resolution, output/figures/)
> 5. Assemble a professional 10-15 page PDF (output/presentation.pdf)
> 6. Generate the requirements.txt
> Mortality data is NOT in scope. Every chart carries one clear
> message, an explicit title and annotations. Sober palette:
> blues, grays, one red/orange accent.
```

Expected inputs:

- Three public datasets (hospital, vaccination, economic) — French open data, file names unchanged

Expected outputs:

- Cleaned data, reusable analysis scripts, publication-ready charts, a 10-15 page narrative PDF deck

> **Companion resources:** replicate this use case at home — the project folder (CLAUDE.md, ready-to-paste prompts, dataset and instructions) is available for download: Google Drive folder. This is the recommended first project of the guide: real public data, no confidentiality constraint, a visible deliverable in one session, and every skill of chapters 3 and 4 exercised at once.

© 2026 Necflow AI & David Schraub Actuarial Consultancy. All rights reserved.

CHAPTER 6

Property & Casualty

In property & casualty, Claude Code helps across the whole chain: pricing, reserving, profitability analysis, reporting and compliance. This section covers the most concrete use cases.

6.1 Pricing

Use case 24 — Auto pricing: GLM, machine learning and variable selection

> **Situation:** In personal auto, a hyper-competitive market, pricing is a major battleground. Generalized linear models remain the standard because they are interpretable and accepted by state regulators in rate filings. But machine learning models offer performance gains and can exploit more variables. Models use numerous variables and go through a variable selection step.

What Claude Code will design: a complete pricing pipeline, from variable selection to the GLM / machine learning comparison, with interpretability.

© 2026 Necflow AI & David Schraub Actuarial Consultancy. All rights reserved.

■ Claude Code Prompt

claude

```
> Build a complete auto pricing pipeline in Python.
>
> DATA: a parquet file with 500,000 policies. Driver variables
> (age, years licensed, prior claims, credit-based insurance score
> where permitted, gender, marital status, territory, vehicle use)
> and vehicle variables (horsepower, vehicle age, make, rating
> group, fuel type, replacement value). Target variables: claim
> count and claim cost.
>
> STEP 1 - Preparation: cleaning, missing value treatment, rare
> modality grouping, continuous variable binning.
>
> STEP 2 - Exploratory analysis: variable distributions, claim
> experience (frequency and average cost) by modality of each
> rating variable, correlations, identification of the most
> discriminating variables. Produce charts and a summary of
> findings before modeling.
>
> STEP 3 - Variable selection: test several approaches (univariate
> analysis, tree-based importance, stepwise selection on the GLM)
> and propose a retained variable set, with justification.
>
> STEP 4 - GLM: a Poisson GLM for frequency, a Gamma GLM for
> average cost. Show relativities by variable and their
> significance.
>
> STEP 5 - Machine learning: train an XGBoost and a neural network
> on frequency and cost, with hyperparameter optimization.
>
> STEP 6 - Comparison: compare GLM and ML on suitable metrics
> (deviance, Gini, lift curves, double lift chart). Use SHAP to
> interpret the ML model and identify segments where the GLM
> under- or over-prices.
>
> STEP 7 - Delivery: a summary report with the charts, the
> segments to watch, and GLM adjustment recommendations drawn
> from the ML findings.
>
> Structure the code in modules, add tests, and comment in
> English.
```

Expected inputs:

- Policy and claims database with driver and vehicle variables

Expected outputs:

- Complete pipeline, frequency-severity GLM, ML models, comparison and SHAP interpretability
- A GLM adjustment recommendation report

> **Note:** Check your state's rules before using certain rating variables: credit-based insurance scores, for example, are restricted or banned in several states. Variable selection is a regulatory question as much as a statistical one.

© 2026 Necflow AI & David Schraub Actuarial Consultancy. All rights reserved.

Note: In practice the number of variables considered is much higher, and each of the steps articulated here requires abundant actuarial judgment. For example, in step 1 the treatment of missing values is currently a hotly debated topic: distinguishing a random omission from a strategic one (a prospect who did not indicate a continuous insurance history may have made an honest mistake, or may be purposefully hiding an insurance gap that would have negative consequences).

Companion resources: replicate this use case at home — the project folder (CLAUDE.md, ready-to-paste prompts, dataset and instructions) is available for download: Google Drive folder. The pack works on freMTPL2freq, a public dataset of 678,000 motor third-party liability policies from the CASdatasets collection, and its prompt also covers the documentation and backtesting of the next use case.

Use case 25 — Documentation, backtesting and the rate filing deck

Situation: A pricing model does not live only on the day it is built. It must be documented for compliance and validation, backtested regularly to check it holds up, and presented to management and to the state insurance department in a rate filing. These three tasks return with every rate revision and consume a lot of time.

What Claude Code will design: from the pricing model — technical documentation, automated backtesting, and a presentation-ready deck.

■ Claude Code Prompt

claude

```
> From my auto pricing model in ./pricing, produce three
> deliverables:
>
> 1. TECHNICAL DOCUMENTATION (Markdown): methodology, retained
>    variables and their justification, GLM relativities,
>    assumptions, model limitations. LaTeX formulas. A style
>    suited to internal validation and to supporting a state rate
>    filing.
>
> 2. BACKTESTING: compare the model's predictions to actually
>    observed claims on a test period. Compute gaps by segment,
>    check the stability of relativities over time, and flag
>    segments where the model drifts. Produce the charts
>    (predicted vs observed, gap evolution by cohort).
>
> 3. PRESENTATION DECK (PowerPoint): about ten slides for
>    management: context, methodology in one slide, key results,
>    segments at stake, estimated rate impact, recommendations.
>    Sober, readable style, one message per slide.
>
> Comment your approach and flag the areas an actuary must
> validate.
```

Expected outputs:

- Pricing model technical documentation, automated backtesting, management deck

© 2026 Necflow AI & David Schraub Actuarial Consultancy. All rights reserved.

6.2 Reserving

Use case 26 — Reserving: Chain Ladder, Bornhuetter-Ferguson, Mack

Situation: P&C reserving rests on classic methods applied to loss development triangles — the same triangles filed in Schedule P. The actuary produces estimates by line of business, with confidence intervals, and analyzes movements against prior evaluations.

What Claude Code will design: a reserving tool that applies several methods to the triangles, computes reserves and confidence intervals, and produces the charts and the export.

■ Claude Code Prompt

claude

```
> Build a P&C reserving tool in Python. It reads loss development
> triangles from an Excel file (one tab per line of business,
> cumulative triangles). It applies three methods: Chain Ladder,
> Bornhuetter-Ferguson (with an a priori loss ratio per line), and
> Mack for confidence intervals.
> For each line, compute: development factors, ultimate losses,
> the IBNR reserve, and the standard error of the estimate (Mack).
> Outputs: a summary table by line and method, the completed
> triangles, charts (development factors, run-off profile, method
> comparison), and an Excel export with conditional formatting.
> Comment in English.
```

Expected inputs:

- Loss triangles by line of business, a priori loss ratios

Expected outputs:

- Reserves by method and line, confidence intervals, charts, Excel export

Companion resources: replicate this use case at home — the project folder (CLAUDE.md, ready-to-paste prompts, dataset and instructions) is available for download: Google Drive folder. The pack ships a ten-year auto claims development triangle and implements the plain Chain Ladder; the prompt above extends it to Bornhuetter-Ferguson and Mack — a good exercise in making a prompt go further than its starting pack.

Companion resources: the extended version of this exact use case — the three methods, ready to run — is also available as its own project folder: Google Drive folder. It carries two ten-year triangles (auto, homeowners) with earned premiums and a priori loss ratios, plus hand-checked reference results for the three methods.

© 2026 Necflow AI & David Schraub Actuarial Consultancy. All rights reserved.

6.3 Performance, Reporting and Compliance

Use case 27 — Profitability analysis and portfolio steering

> **Situation:** The technical division studies portfolio performance (auto, homeowners) and recommends strategic adjustments to management: loss ratio and combined ratio tracking by segment, identification of loss-making segments, drift detection.

What Claude Code will design: a profitability dashboard that decomposes the combined ratio by segment, automatically flags deteriorating segments, and formulates action levers.

■ Claude Code Prompt

```
claude

> Create a profitability analysis tool for a P&C portfolio.
> Inputs: monthly premiums and losses by segment (line, territory,
> distribution channel, customer profile), and expenses.
> Compute by segment: the loss ratio, the expense ratio, the
> combined ratio. Identify segments whose combined ratio exceeds
> 100% and those whose trend has deteriorated over the last 6
> months.
> Produce a dashboard (Streamlit) with: a ratio overview, a
> ranking of the worst segments, a trend analysis, and for each
> problem segment, action levers (rate adjustment, underwriting
> tightening). Add a management report export. Comment in
> English.
```

Expected outputs:

- Profitability dashboard by segment, drift detection, action levers, management report

Use case 28 — Data visualization and exploratory portfolio analysis

> **Situation:** Before any modeling, the P&C actuary must understand the portfolio: variable distributions, claim experience by segment, correlations, outliers. This exploratory analysis is time-consuming and redone for every study.

What Claude Code will design: an automatic exploratory analysis report and a reusable set of visualizations for any P&C portfolio.

© 2026 Necflow AI & David Schraub Actuarial Consultancy. All rights reserved.

■ Claude Code Prompt

```
claude

> Create a Python exploratory analysis module for a P&C
> portfolio. Input: a policy and claims file. Automatically
> produce:
> - a data quality report (missing values, outliers, duplicates,
>   date consistency),
> - the distribution of each variable (histograms, box plots),
> - claim experience by modality of each rating variable
>   (frequency and average cost),
> - a correlation matrix and identification of redundant
>   variables,
> - the main charts exported as PNG plus an HTML summary report.
> The module must be reusable on any portfolio by just changing
> the input file. Comment in English.
```

Expected outputs:

- Data quality report, visualizations, claim experience analysis, HTML report

Use case 29 — Drafting actuarial reports and regulatory watch

> **Situation:** P&C actuaries produce many reports (actuarial report, technical memos, support for the Statement of Actuarial Opinion). Drafting and keeping these documents consistent takes considerable time.

What Claude Code will design: a pre-filled actuarial report frame built from the models' results, and help applying the standards.

■ Claude Code Prompt

```
claude

> From my model results in ./results (reserves, ratios, claim
> experience) and the actuarial report template in
> ./templates/report_frame.docx, generate a draft P&C actuarial
> report:
> - follow the template structure,
> - insert the result tables and charts in the right places,
> - draft the analysis commentary in clear language,
> - add a section on compliance with ASOP 36 (statements of
>   actuarial opinion on reserves) and ASOP 43 (unpaid claim
>   estimates), and a section on NAIC annual statement
>   consistency (Schedule P tie-out),
> - flag the points requiring human validation.
> Produce a Word document. Do not replace my judgment: clearly
> mark the zones to complete or validate. Comment your approach
> in English.
```

Expected outputs:

- A pre-filled actuarial report draft, ASOP sections, Schedule P tie-out, zones flagged for validation

© 2026 Necflow AI & David Schraub Actuarial Consultancy. All rights reserved.

CHAPTER 7

Reinsurance

Reinsurance combines extreme risk modeling, structure optimization, reserve calculations and regulatory compliance. Claude Code helps on each of these fronts.

Use case 30 — Pricing and modeling extreme risks

Situation: The reinsurer assesses extreme risks (natural catastrophes, pandemics) with probabilistic models. Pricing an excess-of-loss treaty requires modeling claim experience beyond a retention, with heavy-tailed distributions.

What Claude Code will design: a pricing tool for an excess-of-loss treaty, with severity distribution fitting and net premium calculation by simulation.

Claude Code Prompt

claude

```
> Build a pricing tool for an excess-of-loss (XL) treaty in
> Python. Inputs: a history of individual claims, the retention
> (attachment point) and the limit of the treaty.
> 1. Fit severity distributions suited to extreme risks
>    (generalized Pareto, lognormal) by maximum likelihood, and
>    compare their goodness of fit (QQ-plots, tests).
> 2. Model the frequency of claims exceeding the retention
>    (Poisson).
> 3. Compute the treaty's net premium by Monte Carlo simulation
>    (100,000 draws): for each simulated year, draw the number of
>    claims and their amounts, apply retention and limit,
>    accumulate the reinsurer's loss.
> 4. Compute the burning cost (the historical average annual loss
>    to the layer), the net premium, the standard deviation and
>    the loss quantiles (VaR, TVaR).
> Outputs: a pricing sheet with fitted parameters, loss
> distribution charts, and the key indicators. Comment in
> English.
```

Expected outputs:

- Fitted severity distributions, net premium, burning cost, VaR/TVaR, pricing sheet

Use case 31 — Optimizing the reinsurance program

Situation: The company must optimize its reinsurance and retrocession program, trading off the cost of cover against the reduction in risk and in required capital. Many structures must be tested and their impact measured.

What Claude Code will design: an optimizer that tests a set of structures (quota share, excess of loss, stop-loss, combinations) and identifies the optimal structure in terms of net cost and capital relief.

© 2026 Necflow AI & David Schraub Actuarial Consultancy. All rights reserved.

■ Claude Code Prompt

```
claude

> Build a reinsurance program optimizer in Python. It tests a set
> of structures: quota share (5% to 40%), excess of loss
> (retention 1M to 5M, limit 5M to 20M), stop-loss, and
> combinations. For each structure, from a simulated loss
> distribution, compute:
> - the net cost of reinsurance (ceded premiums minus expected
>   recoveries),
> - the reduction in loss volatility (standard deviation, TVaR),
> - the estimated reduction in required capital (RBC impact),
> - the impact on expected net income.
> Identify the frontier of efficient structures (minimal cost for
> a target risk level) and recommend a structure. Outputs: a
> comparative table, the efficient frontier chart, and a reasoned
> recommendation. Comment in English.
```

Expected outputs:

- Structure comparison, cost/risk efficient frontier, reasoned recommendation

Use case 32 — Ceded reserves and regulatory reporting

> **Situation:** The reinsurance actuary estimates reserves for assumed and ceded business and produces the indicators required by regulators — including Schedule F, which tracks reinsurance balances and counterparty exposure in the NAIC annual statement for P&C companies.

What Claude Code will design: a tool that computes reinsurance reserves by treaty and generates the expected regulatory indicators.

■ Claude Code Prompt

```
claude

> Create a Python tool that, from the treaty portfolio and ceded
> claim experience, computes reinsurance reserves (ceded case
> reserves, ceded IBNR) by treaty and by accident year. Then
> generate the expected regulatory indicators: ceded balances by
> reinsurer for Schedule F, provisions for reinsurance
> (overdue/collateral logic simplified and documented), and the
> net-to-gross reconciliation. Produce a summary table and an
> export in the format expected by reporting. Comment in
> English.
```

Expected outputs:

- Reinsurance reserves by treaty, Schedule F support, net-to-gross reconciliation, reporting export

© 2026 Necflow AI & David Schraub Actuarial Consultancy. All rights reserved.

CHAPTER 8

Building an Internal Tool in a Day

This is probably the chapter that will resonate most. Beyond scripts and models, Claude Code lets you build genuine internal tools, with an interface, in a fraction of the usual time. Where a project used to go through IT and take weeks, the actuary can now prototype it in a day.

In one day, Claude Code can build:

- **ALM dashboard:** an interactive ALM monitoring dashboard
- **RBC tool:** a risk-based capital calculator
- **ORSA tool:** an ORSA stress test orchestrator
- **Validation tool:** a model validation tool (two-version comparison)
- **Data quality:** a portfolio data quality control tool
- **Report generator:** an automatic statutory report generator
- **Streamlit front-end:** a light web interface to make a model usable by non-technical colleagues

The important point: these tools are not throwaway mockups. Properly structured and tested, they can go to production and be maintained over time.

■ Example: a data quality tool in one session

claude

```
> Create an internal portfolio data quality tool with a Streamlit
> interface. The user drops a contract file (Excel or CSV). The
> tool runs a battery of parametrizable checks: required fields
> populated, consistent dates (effective date < maturity date,
> plausible dates of birth), positive amounts, consistency
> between contract type and coverages, duplicate detection. It
> displays a quality dashboard (compliance rate per check), the
> list of anomalies with row-level detail, and allows exporting
> an anomaly report in Excel. The check rules must be defined in
> an easily editable configuration file. Comment in English and
> provide the launch instructions.
```

> **Note:** The mental shift is here: you no longer wait for an IT development to get a tool. You describe the need, you get a working prototype within the day, and you iterate with the users.

© 2026 Necflow AI & David Schraub Actuarial Consultancy. All rights reserved.

CHAPTER 9

Building AI Agents with Claude Code

Claude Code is not only for writing code: it can also help you build AI agents — systems that accomplish tasks autonomously by chaining actions and using tools. For an actuary, this is a new category of internal tools, and arguably the most promising.

9.1 What an Agent Is, Concretely

An agent is a program that receives an objective, breaks the task down, calls tools (file reading, database queries, calculations, language model calls), observes the results and decides the next step, until the objective is reached. Where a script follows a fixed sequence, an agent adapts. Concretely, for an actuary, an agent can navigate a documentation base, understand a business question asked in plain English, find the right information in the specifications or the code, and formulate a reasoned answer.

9.2 Two Families of Particularly Useful Agents

The first: an agent with access to all the functional specifications that answers questions. An actuarial model's specifications often span hundreds of pages across many documents. Finding the precise treatment of a particular case can take an hour. An agent that indexes all the specifications answers in seconds, citing the exact passage.

The second, more powerful: an agent with access to a team's entire codebase that answers business questions to evolve or correct the specifications. A concrete example: a life modeling team uses a cash flow projection tool. The agent has access to that tool's code. When a business team wonders how a given flow is actually calculated, or wants to evolve a specification, the agent reads the code, explains what is really implemented, spots gaps against the written specification, and helps correct erroneous specs.

9.3 Other Agents Useful to an Actuary

- **Monitoring agent:** an agent that watches data quality at each new portfolio delivery and alerts on anomalies
- **Reporting agent:** an agent that, from a model's results, drafts a first analysis memo and formats it
- **Control agent:** an agent that checks consistency between a technical memo and the tables actually used in the model

© 2026 Necflow AI & David Schraub Actuarial Consultancy. All rights reserved.

Prompt — Agent answering from all the specifications

■ Claude Code Prompt

claude

```
> Help me build a Python agent that answers business questions
> from my full set of functional specifications (folder ./specs,
> multiple Word and PDF documents). The agent must:
> 1. Index all the documents (split into sections, build a
>    semantic search index).
> 2. Receive a question in plain English (e.g.: how are scheduled
>    partial withdrawals treated?).
> 3. Retrieve the relevant passages from the specifications.
> 4. Formulate a clear answer citing precisely the source document
>    and section, without inventing: if the information is not in
>    the specifications, the agent must say so.
> Propose the architecture (indexing, search, answer generation)
> before implementing. Provide a simple command-line interface.
> Comment in English.
```

Companion resources: replicate this use case at home — the project folder (CLAUDE.md, ready-to-paste prompts, dataset and instructions) is available for download: Google Drive folder. The pack goes further than this prompt: a complete Streamlit chat application with local embeddings and a ChromaDB index over a sample documentation-and-code folder — ready to point at your own files.

Prompt — Agent answering from a team's code

■ Claude Code Prompt

claude

```
> Help me build a Python agent that has access to the code of a
> cash flow projection tool (flows received and paid by the
> insurer), in ./flow_tool, and that answers business teams'
> questions to evolve the specifications. The agent must:
> 1. Explore and understand the code structure (modules, flow
>    calculation functions, parameters).
> 2. Receive a business question (e.g.: how is the dividend flow
>    calculated in the tool? does it match the specification in
>    ./specs/dividend_spec.docx?).
> 3. Read the relevant code, explain in clear language what is
>    actually implemented, and compare it to the written
>    specification.
> 4. Flag gaps between code and specification, and propose either
>    a code fix or a specification fix.
> The agent must always cite the files and code lines it relies
> on. Explain the architecture before implementing. Comment in
> English.
```


© 2026 Necflow AI & David Schraub Actuarial Consultancy. All rights reserved.

Prompt — Data quality monitoring agent

■ Claude Code Prompt

claude

```
> Help me build a Python agent that monitors data quality at each
> new portfolio delivery. For every new file dropped in
> ./deliveries, the agent must: run a battery of checks
> (completeness, consistency, bounds, duplicates, comparison with
> the previous delivery to detect abnormal variations), produce a
> quality report, and send a synthetic alert if blocking
> anomalies are detected. The check rules must be externalized in
> a configuration file. Explain the architecture before
> implementing. Comment in English.
```

Oversight: An agent acting autonomously must always remain under human supervision for decisions with stakes. In actuarial work, the agent prepares, proposes and alerts; the actuary validates.

© 2026 Necflow AI & David Schraub Actuarial Consultancy. All rights reserved.

CHAPTER 10

Security, Confidentiality and AI Governance

Insurance data is among the most sensitive there is: personal data, health data, financial data. Using an AI tool in this context demands clear rules. This chapter is not optional: it conditions the very possibility of using these tools in a regulated professional setting. It also reflects a reality of the US market: regulators are actively building expectations for insurers' use of AI.

The right question, and who answers it

This guide will not hand you a confidentiality rulebook, because the honest answer is simpler: the modalities under which you may use Claude Code on professional data are set by your company, not by a book. Before processing anything beyond public or synthetic data, talk to your IT security and compliance teams and ask the one question that matters: under which deployment, and on which classes of data, is the tool approved here?

The reason the question deserves a real answer is that most of what an actuary produces — rate filings, regulatory reports, valuation memos — is confidential by nature. Keeping all of it away from the tool would take away most of the tool's usefulness; the working answer in many companies is a deployment that has been locked away from communication with the outside world and validated by IT security, so that confidential work can happen inside a controlled perimeter. What that perimeter looks like — enterprise plan with contractual no-training guarantees, dedicated environment, scoped accounts — is exactly what your IT department will tell you.

On an individual plan, the account-level setting shown in chapter 3 (turning off “Help improve our AI models”) controls whether your conversations and coding sessions may be used for model training. Review it before your first session; in a corporate setting it complements, and never replaces, your company's own controls.

Whatever the deployment, three reflexes cost nothing and always apply:

- **Secrets:** never paste secrets (passwords, API keys, connection strings) into a prompt. Use environment variables.
- **Review before execution:** have generated code reviewed before running it on real data, especially if it writes, moves or deletes files.
- **Logging:** log the actions of tools and agents so you can audit what was done.

© 2026 Necflow AI & David Schraub Actuarial Consultancy. All rights reserved.

What to remember

AI changes how you work, not your responsibility framework. An actuary using Claude Code is held to the same standards of rigor, documentation and compliance as if everything had been written by hand. Good practice treats AI-produced code and analyses exactly like a junior colleague's: useful, fast, but systematically reviewed and validated before anything is committed.

> **Golden rule:** Use AI to automate execution, not accountability. AI accelerates the work; it does not dilute responsibility. The actuary remains accountable for the models, the results and their compliance.

© 2026 Necflow AI & David Schraub Actuarial Consultancy. All rights reserved.

CHAPTER 11

Best Practices and the Correction Workflow

11.1 Prompting Claude Code Well

- **Precision:** be precise about inputs, outputs and the expected format.
- **Formulas:** provide exact mathematical formulas when they exist.
- **Tests:** ask for unit tests together with the code.
- **Conventions:** put your recurring conventions in the CLAUDE.md file.
- **Chunking:** break complex tasks down: ask for the architecture, validate, then the implementation.
- **Context:** ask Claude Code to read the existing code before modifying it.

Some best practice techniques for prompts

- **ROTA — Role • Objective • Task • Audience • Constraints.** Defines who the AI is, what success looks like, the specific job, who the output is for, and any boundaries. Reduces ambiguity and improves relevance. Works especially well for business writing and analysis; produces more targeted, audience-appropriate responses.
- **CRISP — Context • Request • Instructions • Style • Parameters.** Starts with background information before stating the request, adds explicit instructions and the desired communication style, and uses parameters such as length, format and tone. Creates consistent, high-quality outputs across tasks.
- **CARE — Context • Action • Result • Evaluation.** Provides the situation and the action the AI should take, specifies the desired outcome or deliverable, and includes evaluation criteria for self-checking quality. Useful for problem solving, reports and decision support.
- **SMART — Specific • Measurable • Actionable • Relevant • Time-bound.** Frames prompts with clear goals and success metrics, encourages practical, implementation-focused responses, and keeps outputs aligned to a defined objective. Excellent for planning, project management and productivity.
- **COPE — Context • Objective • Process • Examples.** Explains the background and desired end state, requests a step-by-step reasoning process, and provides examples of the preferred output format. Especially effective for complex analytical or technical tasks.

11.2 The Correction Loop

This is the mechanism that impresses most the first time you see it. Facing a failing test, Claude Code does not stop: it diagnoses and corrects on its own.

- **Step 1:** you launch the tests (or ask Claude Code to launch them).
- **Step 2:** a test fails. Claude Code reads the error message in the console.
- **Step 3:** it identifies the cause, proposes a fix and applies it.
- **Step 4:** it re-runs the tests to check the problem is solved, without regression.

© 2026 Necflow AI & David Schraub Actuarial Consultancy. All rights reserved.

- **Step 5:** you validate the final result.

11.3 A Trick That Changes Everything: Synthetic Datasets

To develop without exposing real data, ask Claude Code to generate a realistic synthetic dataset (same structure, same orders of magnitude, no personal data). You develop and test on it, then run the validated code on the real data yourself.

11.4 A Complete Example: the Quarterly Portfolio Review Pipeline

Here is a workflow we use in training and live demonstrations — including the Society of Actuaries webcast this guide draws on. It chains four prompts on a 500,000-policy portfolio, each step feeding the next. It is the level-3 pattern from chapter 2, applied to a real actuarial close.

- **Step 1 — Data quality assessment.** Claude Code audits the raw extract: missing values, duplicate policy numbers, impossible ages, negative face amounts, death claims without a date of death. It flags and quantifies — it fixes nothing. Deliverables: an exception report (CSV), a summary by issue type, and the Python script that produced them.
- **Step 1b — Apply the cleaning decisions.** The actuary reviews the report and decides each treatment. A second prompt applies those decisions and produces a clean dataset plus a cleaning log (records affected per rule). The decision stays human; the execution is automated.
- **Step 2 — Exploratory analysis.** On the clean dataset only: portfolio composition, exposure concentrations, lapse rates by duration, crude mortality by age and gender. Deliverables: publication-ready charts (PNG), segmentation tables (CSV), and the analysis script.
- **Step 3 — Automated reporting.** Without re-running any computation, Claude Code assembles the step-2 outputs into two PDF reports: a technical report for the actuarial team and a two-page executive summary for management. Same analysis, two audiences, two deliverables.

> **Tip:** Every step saves its own Python script next to its outputs. Next quarter, the same scripts run on the new extract and the whole close reproduces itself. Reproducibility is the real deliverable.

> **Companion resources:** replicate this use case at home — the project folder (CLAUDE.md, ready-to-paste prompts, dataset and instructions) is available for download: Google Drive folder. The pack contains the full project: the CLAUDE.md, the four prompts, and the 500,000-policy synthetic portfolio with its planted data issues.

11.5 Going Further: Migrating the Whole Study to R

A team that works in R does not have to redo the quarterly review from scratch. Code migration is one of the tasks Claude Code handles best, because the specification already exists: the source code itself, and its outputs. The approach works in three prompts, each producing its own deliverable:

© 2026 Necflow AI & David Schraub Actuarial Consultancy. All rights reserved.

■ Claude Code Prompt

claude

```
> 1. INVENTORY: read every Python script in ./quarterly_review and
>    its outputs. Produce a migration plan: scripts, dependencies,
>    the R equivalents (tidyverse, data.table) and the points that
>    need care (joins, date handling, floating-point aggregates).
>
> 2. MIGRATE: translate each script to R, same inputs, same file
>    outputs, same folder structure. Comment in English.
>
> 3. COMPARE: run both versions on the same extract and produce a
>    reconciliation report — every figure and every CSV compared,
>    differences explained or fixed. The migration is done when
>    the reconciliation is exact.
```

Note: The reconciliation step is what makes the migration auditable: you do not trust the translation, you verify its outputs against the original's.

Companion resources: replicate this use case at home — the project folder (CLAUDE.md, ready-to-paste prompts, dataset and instructions) is available for download: Google Drive folder. The pack contains a worked Python-to-R migration of the quarterly-review scripts, with the reconciliation outputs.

© 2026 Necflow AI & David Schraub Actuarial Consultancy. All rights reserved.

CHAPTER 12

Appendix — A CLAUDE.md Template for Actuaries

Here is a CLAUDE.md template designed for an actuary. Copy it to the root of your project and adapt it to your environment. This file gives Claude Code the context it needs to produce code matching your expectations from the first iteration.

CLAUDE.md

```
# Project context
Actuarial model - [Life / P&C / Reinsurance]
Objective: [ALM projection / reserving / pricing / ...]
# Technical stack
- Python 3.11: pandas, numpy, scipy, statsmodels
- Machine learning: scikit-learn, xgboost, shap
- Visualization: matplotlib, plotly, streamlit
- Legacy models in Excel/VBA (folder ./legacy)
- Data: parquet format, monthly time step
# Code conventions
- Variable names and comments in English
- Docstrings for every function (role, inputs, outputs, units)
- Systematic unit tests (pytest) for every calculation function
- Variable naming per the internal actuarial standard
# Business rules
- Always check cash flow consistency before validating
- Discount with the prescribed statutory rates unless told
  otherwise
- Default mortality: 2017 CSO, age nearest birthday (ANB),
  gender- and smoker-distinct, select and ultimate
- Every implicit assumption must be documented in the code
# Security and confidentiality
- Never use real personal data: synthetic data only
- Never write secrets in the code
- Flag any code that writes or deletes files before execution
# What I expect from you
- Propose the architecture before implementing large modules
- Write tests at the same time as the code
- Explain modeling choices in plain English
- Flag anything that requires actuarial validation
```

Tip: Enrich this file over time. Every time you correct the same thing twice, add a rule to the CLAUDE.md: you will never correct it again.

© 2026 Necflow AI & David Schraub Actuarial Consultancy. All rights reserved.

About the Authors



Ethel Cédric Nzati, AQ

Lead author — Actuary & Data Scientist

- Life insurance actuary (ALM), ENSAE Paris graduate
- AI content creator: YouTube (4,700+ subscribers), LinkedIn (10,000+ followers)
- Founder, Necflow AI — AI solutions and automation for enterprises
- Author of “Claude Code pour les Actuaires”, the French-language reference guide
- Presented the “Claude Code for Actuaries” webcast for the Society of Actuaries (October 2026)
- Runs L'Accélérateur IA, a 1,200+ member community on applied AI



David Schraub, FSA, CERA, MAAA, AQ

Co-author — Independent Actuary, AI & Model Governance

- Independent actuary specializing in AI and model governance
- Expertise in bias and fairness testing for actuarial models
- Former Research Actuary at the Society of Actuaries (technology and innovation)
- Career spanning consulting firms, direct carriers and the SOA Research Institute
- Active volunteer: SOA, American Academy of Actuaries, IAA, Institut des actuaires
- French and American credentialled actuary

YouTube	youtube.com/@ethelcedric
LinkedIn (Ethel)	linkedin.com/in/ethelnzati
LinkedIn (David)	linkedin.com/in/davidschraub
Necflow AI	necflow.com
David Schraub Actuarial Consultancy	davidschraubactuary.com

Disclaimer: GenAI has been leveraged to write this book, for phrasing and grammar verification. However, every sentence has been reviewed by one of the authors.

© 2026 Necflow AI & David Schraub Actuarial Consultancy. All rights reserved.

© 2026 Necflow AI & David Schraub Actuarial Consultancy. All rights reserved.